ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

LÝ THUYẾT ĐIỀU KHIỂN NÂNG CAO

Báo cáo Bài tập lớn

Điều khiển mô hình trượt robot bốn chân sử dụng ROS/Matlab Simulink trong môi trường Webots

GVHD: PGS.TS Nguyễn Tấn Lũy

SV thực hiện: Trần Đặng Khánh Sơn 2312979

Nguyễn Thị Huyền Trân 2313555

Dư Bách Thư 2313377

TP. Hồ Chí Minh, tháng 5/2026

Danh sách hình vẽ

1 Mô hình robot bốn chân trong môi trường mô phỏng Webots 2
2 Sơ đồ động học 1 chân của robot 4
3 Sơ đồ khối bộ điều khiển trượt 9
4 Đồ thị hàm sat $\left(\frac{s}{\Phi}\right)$ và hàm $\mathrm{sgn}(s)$ 10
5 Biểu đồ pha của các chân với dáng đi Trot 12
6 Quỹ đạo điểm cuối của chân 13
7 Lưu đồ các bước thiết lập và kiểm chứng mô phỏng trên Simulink 14
8 Mô hình Simulink kiểm chứng mô hình động lực học và thuật toán SMC cho 1 chân . . . 15
9 Khối tạo quỹ đạo 15
10 Bộ điều khiển SMC 15
11 Đối tượng điều khiển 16
12 Hệ thống con cho 1 chân 16
13 Khối lập lịch dáng đi 16
14 Mô hình Simulink kiểm chứng mô hình động lực học và thuật toán SMC cho 1 chân . . . 17
15 So sánh đáp ứng bám quỹ đạo của bộ điều khiển SMC trước và sau khi tối ưu thông số . 18
16 Mô hình Simulink kiểm chứng mô hình động lực học và thuật toán SMC cho 1 chân . . . 19
17 Quỹ đạo điểm cuối của chân 21
18 Lưu đồ giải thuật của hệ thống 22
19 Bảng điều khiển gait từ bàn phím 25
20 Bảng theo dõi sai số bám của từng khớp 26
21 Đáp ứng bám quỹ đạo của dáng đi Walk 31
22 Đáp ứng bám quỹ đạo của dáng đi Bound 32
23 Sai số bám của dáng đi Bound 33
24 Đáp ứng bám quỹ đạo của dáng đi Walk 33

Mục lục

LỜI MỞ ĐẦU **1**

Chương 1: MÔ HÌNH HÓA VÀ ĐỘNG LỰC HỌC **2**
1.1 Xây dựng mô hình 3D trong Webots . . . 2
1.1.1 Cấu trúc tổng quan . . . 2
1.1.2 Cấu hình khớp và thông số vật lý của các khâu . . . 2
1.1.3 Hệ thống cảm biến và bộ tryền động . . . 3
1.1.4 Thông số môi trường và tiếp xúc . . . 3
1.2 Phân tích động học khâu chân . . . 4
1.2.1 Thiết lập hệ trục tọa độ và thông số hình học . . . 4
1.2.2 Phương trình động học thuận . . . 5
1.2.3 Phương trình động học ngược . . . 5
1.3 Phân tích động học khâu chân . . . 6
1.3.1 Thiết lập thông số động lực học ban đầu . . . 6
1.3.2 Động năng, thế năng và hàm Lagrangian . . . 6
1.3.3 Phương trình Euler-Lagrange và Ma trận Động lực học chuẩn . . . 7

Chương 2: THIẾT KẾ BỘ ĐIỀU KHIỂN TRƯỢT (SMC) **9**
2.1 Thiết lập mặt trượt . . . 9
2.2 Thành phần điều khiển bù động lực học . . . 9
2.3 Thành phần chuyển mạch và giải pháp chống Chattering . . . 10
2.4 Tích hợp khâu tích phân và ràng buộc thực tế . . . 10
2.5 Chứng minh ổn định Lyapunov . . . 11

Chương 3: QUY HOẠCH DÁNG ĐI VÀ QUỸ ĐẠO **12**
3.1 Thiết kế các dáng đi . . . 12
3.2 Quỹ đạo chân . . . 12

Chương 4: MÔ PHỎNG VÀ KIỂM CHỨNG TRÊN MATLAB/SIMULINK **14**
4.1 Xây dựng mô hình mô phỏng trên Simulink . . . 14
4.1.1 Kiểm chứng động lực học và SMC trên nhánh chân đơn lẻ . . . 15
4.1.2 Đóng gói hệ thống và tích hợp quỹ đạo Trot cho bốn chân . . . 16
4.2 Kết quả kiểm chứng thuật toán . . . 17
4.2.1 Đáp ứng của nhánh chân đơn lẻ với quỹ đạo thử nghiệm . . . 17
4.2.1 Đáp ứng của hệ thống với dáng đi Trot . . . 19

Chương 5: TRIỂN KHAI HỆ THỐNG TRÊN ROS VÀ WEBOTS **21**
5.1 Phát triển bộ điều khiển và tương tác (Webots Standalone) . . . 21
5.1.1 Kiến trúc hệ thống . . . 21
5.1.2 Quản lý luồng tín hiệu và lọc dữ liệu . . . 21
5.1.3 Xây dựng máy trạng thái và tương tác . . . 22
5.2 Tích hợp hệ thống qua ROS . . . 23
5.2.1 Tổ chức gói phần mềm ROS2 (Package Structure) . . . 23
5.2.2 Kiến trúc các Node và vai trò . . . 24

5.2.3 Hệ thống biên dịch với *ament_cmake* 26
5.2.4 Luồng dữ liệu và vòng điều khiển 27
5.2.5 Hệ thống Launch và khởi động tích hợp 28
5.2.6 Cơ chế chuyển đổi dáng đi và xử lý sự kiện 29
5.2.7 Tổng kết kiến trúc hệ thống 29

Chương 6: KẾT QUẢ VÀ THẢO LUẬN **31**
6.1 Kết quả bám quỹ đạo bàn chân 31
6.1.1 Dáng đi Trot (Đi nước kiệu) 31
6.1.2 Dáng đi Bound (Phi ngựa) 31
6.1.1 Dáng đi Walk (Đi bộ) 33

LỜI KẾT **35**

TÀI LIỆU THAM KHẢO **36**

LỜI MỞ ĐẦU

1 Tính cấp thiết của đề tài

2 Mục tiêu

3 Quá trình thực hiện dự án

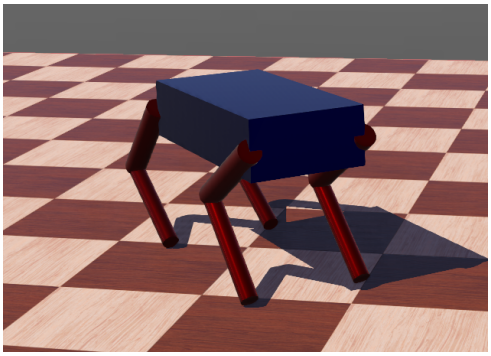
4 Tổng quan về báo cáo

Chương 1: MÔ HÌNH HÓA VÀ ĐỘNG LỰC HỌC

1.1 Xây dựng mô hình 3D trong Webots

1.1.1 Cấu trúc tổng quan

Mô hình robot bốn chân được xây dựng trong môi trường mô phỏng vật lý Webots, sử dụng định dạng VRML (Virtual Reality Modeling Language). Robot có kiến trúc đối xứng gồm một thân cứng trung tâm và bốn chân, mỗi chân có 2 bậc tự do, tổng cộng 8 bậc tự do (DOF).



Hình 1: *Mô hình robot bốn chân trong môi trường mô phỏng Webots*

Thân robot được mô hình hóa là một khối hộp chữ nhật (Box) với kích thước 0.4 m × 0.2 m × 0.1 m (dài × rộng × cao), khối lượng 5 kg, màu xanh navy. Bốn chân được đặt tên theo vị trí: FL (Front-Left – Trước Trái), FR (Front-Right – Trước Phải), BL (Back-Left – Sau Trái) và BR (Back-Right – Sau Phải), gắn đối xứng tại bốn góc thân theo Bảng 1.

Bảng 1: *Vị trí gắn chân vào thân robot*

Chân	Vị trí gắn vào thân (x, y, z) [m]
FL	$(+0.2,\ +0.1,\ 0)$
FR	$(+0.2,\ -0.1,\ 0)$
BL	$(-0.2,\ +0.1,\ 0)$
BR	$(-0.2,\ -0.1,\ 0)$

1.1.2 Cấu hình khớp và thông số vật lý của các khâu

Mỗi chân robot gồm hai khâu được nối bởi hai khớp bản lề (`HingeJoint`), đều xoay quanh trục Y (axis = [0, 1, 0]), tương ứng với chuyển động trong mặt phẳng dọc (xOz).

a) Khớp hông (Hip Joint):

- Phạm vi góc: $q_{\text{hip}} \in [-1.5,\ +1.5]$ rad.
- Khâu đùi: hình trụ , chiều cao $L_1 = 0.2$ m, bán kính 0.02 m.
- Khối lượng đùi: 1 kg.

b) Khớp gối (Knee Joint):

- Điểm neo: cách gốc khớp hông 0.1 m theo trục Z âm.
- Phạm vi góc: $q_{\text{knee}} \in [-2.5,\ 0]$ rad (chỉ gập về phía sau).

- Khâu cẳng chân: hình trụ , chiều cao $L_2 = 0.2$ m, bán kính 0.015 m.
- Khối lượng cẳng chân: 0.5 kg.

Tổng khối lượng robot được tính như sau:

$$m_{\text{tổng}} = m_{\text{thân}} + 4 \times (m_{\text{đùi}} + m_{\text{cẳng chân}}) = 5 + 4 \times (1 + 0.5) = 11 \text{ kg} \tag{1}$$

Bảng 2 tổng hợp thông số vật lý của toàn bộ mô hình.

Bảng 2: *Thông số vật lý của mô hình robot*

Bộ phận	Hình dạng	Kích thước chính	Khối lượng
Thân	Box	$0.4 \times 0.2 \times 0.1$ m	5.0 kg
Đùi (×4)	Cylinder	$h = 0.2$ m, $r = 0.02$ m	1.0 kg
Cẳng chân (×4)	Cylinder	$h = 0.2$ m, $r = 0.015$ m	0.5 kg

1.1.3 Hệ thống cảm biến và bộ tryền động

a) Cảm biến đo góc khớp (Encoders):

Mỗi khớp được trang bị một cảm biến vị trí góc (`PositionSensor`). Tổng cộng 8 encoder cung cấp phản hồi góc khớp tức thời q_i cho bộ điều khiển, được liệt kê trong Bảng 3.

Bảng 3: *Danh sách cảm biến góc khớp*

Tên cảm biến	Khớp tương ứng
`sensor_FL_hip`	Hông – FL
`sensor_FL_knee`	Gối – FL
`sensor_FR_hip`	Hông – FR
`sensor_FR_knee`	Gối – FR
`sensor_BL_hip`	Hông – BL
`sensor_BL_knee`	Gối – BL
`sensor_BR_hip`	Hông – BR
`sensor_BR_knee`	Gối – BR

b) Cảm biến quán tính (IMU – InertialUnit):

Một cảm biến IMU được gắn trực tiếp lên thân robot (tên `"imu"`), cung cấp dữ liệu góc Euler (Roll, Pitch, Yaw) theo thời gian thực. Dữ liệu này được sử dụng để giám sát độ ổn định tư thế thân robot trong suốt quá trình di chuyển (xem Chương 6).

c) Bộ truyền động (RotationalMotors):

Mỗi khớp được dẫn động bởi một `RotationalMotor` tương ứng. Tất cả 8 motor đều có thông số mô-men xoắn cực đại:

$$\tau_{\max} = 100 \text{ N·m} \tag{2}$$

và hoạt động ở chế độ *torque control* – xuất mô-men trực tiếp từ bộ điều khiển SMC. Danh sách motor bao gồm: `motor_FL_hip`, `motor_FL_knee`, `motor_FR_hip`, `motor_FR_knee`, `motor_BL_hip`, `motor_BL_knee`, `motor_BR_hip`, `motor_BR_knee`.

1.1.4 Thông số môi trường và tiếp xúc

Môi trường mô phỏng được cấu hình với bước thời gian tích phân:

$$\Delta t = 2 \text{ ms} \quad (\texttt{basicTimeStep} = 2) \tag{3}$$

Đặc tính ma sát giữa chân robot và nền sàn được mô hình hóa theo mô hình Coulomb với hệ số:

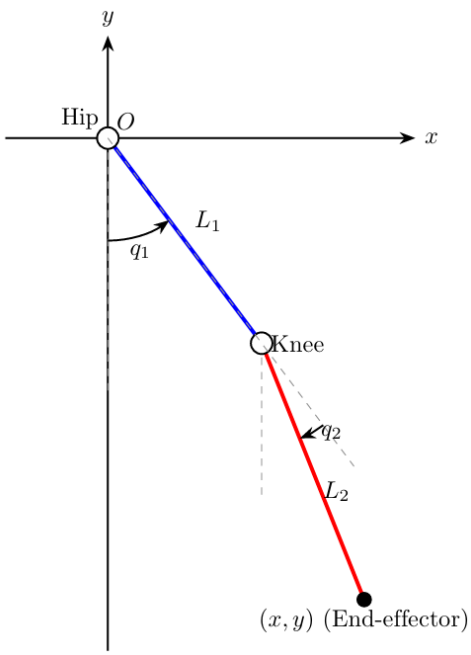
$$\mu = 3 \quad \text{(giữa vật liệu "solid" và "floor")} \tag{4}$$

Hệ số ma sát cao này mô phỏng bề mặt có độ bám tốt, hạn chế hiện tượng trượt chân trong quá trình chuyển động, đảm bảo tính nhất quán với giả thiết "không trượt" trong bài toán điều khiển. Bộ điều khiển phía ngoài kết nối với mô hình thông qua chế độ `controller = "<extern>"`, cho phép các thuật toán SMC được phát triển độc lập (C++/ROS) tương tác trực tiếp với Webots thông qua Webots API.

1.2 Phân tích động học khâu chân

Mỗi chân robot được mô hình hóa như một chuỗi động học hở gồm hai bậc tự do (2-DOF) hoạt động trong mặt phẳng dọc. Mục tiêu là thiết lập mối quan hệ toán học giữa không gian khớp và không gian công tác.

1.2.1 Thiết lập hệ trục tọa độ và thông số hình học



Hình 2: *Sơ đồ động học 1 chân của robot*

Gốc tọa độ được đặt tại khớp hông của mỗi chân. Trục Ox hướng về phía trước theo chiều chuyển động của robot, trục Oy hướng thẳng đứng lên trên. Như hình 2, cấu trúc chân bao gồm hai khâu chính:

- L_1: Chiều dài khâu đùi (Thigh length).
- L_2: Chiều dài khâu cẳng chân (Calf length).
- q_1: Góc khớp hông (Hip joint angle). Quy ước $q_1 = 0$ khi chân duỗi thẳng hướng xuống dọc theo phần âm của trục Oy.
- q_2: Góc khớp gối (Knee joint angle).
- (x, y): Tọa độ của điểm cuối bàn chân (End-effector) trong hệ quy chiếu gốc hông.

1.2.2 Phương trình động học thuận

Động học thuận xác định vị trí điểm cuối bàn chân (x, y) khi biết trước các biến khớp (q_1, q_2). Dựa trên quy ước hệ trục tọa độ (với $q_1 = 0$ thì chân hướng theo góc $-\frac{\pi}{2}$ so với trục Ox), ta có phương trình hình học:

$$\begin{cases} x = L_1 \cos\left(q_1 - \frac{\pi}{2}\right) + L_2 \cos\left(q_1 + q_2 - \frac{\pi}{2}\right) \\ y = L_1 \sin\left(q_1 - \frac{\pi}{2}\right) + L_2 \sin\left(q_1 + q_2 - \frac{\pi}{2}\right) \end{cases} \tag{5}$$

Phương trình động học thuận là cơ sở để kiểm chứng sự chính xác của bài toán động học ngược và mô phỏng không gian làm việc của robot.

1.2.3 Phương trình động học ngược

Để điều khiển bàn chân bám theo tọa độ (x, y) mong muốn, ta cần giải bài toán động học ngược để tìm ra bộ góc khớp (q_1, q_2). Bài toán được giải quyết thông qua phương pháp tiếp cận hình học.

Xử lý điểm kỳ dị:

Trước khi tính toán, khoảng cách từ gốc tọa độ đến bàn chân $L = \sqrt{x^2 + y^2}$ được kiểm tra để đảm bảo không vượt quá giới hạn vật lý của chân.

$$L_{\max} = L_1 + L_2 - \epsilon \tag{6}$$

Với ϵ là một hệ số bù nhỏ ($\epsilon = 0.001$). Nếu hệ thống yêu cầu một tọa độ vượt ngoài $L_{\max}$, tọa độ (x, y) sẽ được thu phóng lại để đảm bảo tính ổn định số học cho hệ phương trình.

Bước 1: Tính toán góc khớp gối q_2

Áp dụng định lý hàm số Cosin cho tam giác tạo bởi các khâu L_1, L_2 và đoạn thẳng nối từ gốc đến bàn chân:

$$L^2 = L_1^2 + L_2^2 + 2L_1L_2\cos(q_2) \tag{7}$$

Từ đó suy ra:

$$\cos(q_2) = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2} \tag{8}$$

Để đảm bảo robot di chuyển với cấu hình đầu gối gập về phía sau (cấu hình đặc trưng của robot bốn chân), giá trị của q_2 được chọn mang dấu âm:

$$q_2 = -\arccos(\cos(q_2)) \tag{9}$$

Bước 2: Tính toán góc khớp hông q_1

Góc khớp hông được xác định bằng cách xét hiệu số giữa hai góc α và β, trong đó:

- α là góc tạo bởi vector vị trí bàn chân và trục Ox:

$$\alpha = \text{atan2}(y, x) \tag{10}$$

- β là góc lệch hình học bên trong cơ cấu chân, được tính bằng:

$$\beta = \text{atan2}(L_2 \sin(q_2), L_1 + L_2 \cos(q_2)) \tag{11}$$

Cuối cùng, góc khớp hông được hiệu chỉnh thêm một lượng $\frac{\pi}{2}$ để khớp với quy ước gốc tọa độ ban đầu:

$$q_1 = (\alpha - \beta) + \frac{\pi}{2} \tag{12}$$

Thông qua hệ phương trình này, mọi tọa độ (x, y) hợp lệ đều có thể được ánh xạ chính xác sang không gian khớp, tạo tiền đề cho quá trình quy hoạch quỹ đạo ở các bước tiếp theo.

1.3 Xây dựng phương trình Động lực học theo phương pháp Euler-Lagrange

Mục tiêu của phần này là thiết lập phương trình động lực học tổng quát cho một nhánh chân robot (2 bậc tự do) hoạt động trên mặt phẳng dọc. Phương pháp Euler-Lagrange được sử dụng để xây dựng mô hình toán học, từ đó trích xuất các ma trận động lực học cốt lõi (M, C, G) phục vụ cho việc thiết kế bộ điều khiển trượt (SMC) ở Chương 2.

1.3.1 Thiết lập thông số động lực học ban đầu

Mặc dù mỗi khâu của robot có ma trận quán tính 3D đặc trưng, chuyển động của nhánh chân được giới hạn trong mặt phẳng dọc (Planar Motion). Do đó, ta chỉ xét thành phần mô-men quán tính chính quanh trục Z vuông góc với mặt phẳng chuyển động. Dựa trên thiết kế từ môi trường mô phỏng Webots, các thông số vật lý được xác định như sau:

- **Khâu 1 (Đùi):** Khối lượng $m_1 = 1.0$ kg, chiều dài $L_1 = 0.2$ m, bán kính $r_1 = 0.02$ m, khoảng cách từ khớp háng đến khối tâm $L_{c1} = 0.1$ m.
  Mô-men quán tính: $I_1 = \frac{1}{12} m_1 (3r_1^2 + L_1^2) \approx 0.003433$ kg.m^2.
- **Khâu 2 (Cẳng chân):** Khối lượng $m_2 = 0.5$ kg, chiều dài $L_2 = 0.2$ m, bán kính $r_2 = 0.015$ m, khoảng cách từ khớp gối đến khối tâm $L_{c2} = 0.1$ m.
  Mô-men quán tính: $I_2 = \frac{1}{12} m_2 (3r_2^2 + L_2^2) \approx 0.001695$ kg.m^2.

1.3.2 Động năng, thế năng và hàm Lagrangian

Tổng động năng (K): Là tổng năng lượng tịnh tiến và năng lượng quay của khâu đùi (K_1) và khâu cẳng chân (K_2):

$$K_1 = \frac{1}{2} m_1 L_{c1}^2 \dot{q}_1^2 + \frac{1}{2} I_1 \dot{q}_1^2 \tag{13}$$

$$K_2 = \frac{1}{2} m_2 \left(L_1^2 \dot{q}_1^2 + L_{c2}^2 (\dot{q}_1 + \dot{q}_2)^2 + 2 L_1 L_{c2} \dot{q}_1 (\dot{q}_1 + \dot{q}_2) \cos q_2 \right) + \frac{1}{2} I_2 (\dot{q}_1 + \dot{q}_2)^2 \tag{14}$$

Tổng động năng của hệ thống: $K(q, \dot{q}) = K_1 + K_2$.

Tổng thế năng (P): Được tính dựa vào độ cao của khối tâm so với trục ngang, với gia tốc trọng trường $g = 9.81$ m/s^2 tác dụng theo chiều âm của trục Y:

$$P_1 = m_1 g y_1 = m_1 g L_{c1} \sin q_1 \tag{15}$$

$$P_2 = m_2 g y_2 = m_2 g (L_1 \sin q_1 + L_{c2} \sin(q_1 + q_2)) \tag{16}$$

Tổng thế năng của hệ thống: $P(q) = P_1 + P_2$.

Hàm Lagrangian (L): Được định nghĩa là hiệu số giữa tổng động năng và tổng thế năng của hệ thống:

$$L(q, \dot{q}) = K(q, \dot{q}) - P(q) \tag{17}$$

1.3.3 Phương trình Euler-Lagrange và Ma trận Động lực học chuẩn

Áp dụng phương trình Euler-Lagrange cho từng biến khớp q_i ($i = 1, 2$ tương ứng với khớp háng và khớp gối) để tìm lực xoắn τ_i tác động lên các khớp:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i} = \tau_i \tag{18}$$

Sau khi thực hiện các phép đạo hàm và phân loại các số hạng theo biến gia tốc ($\ddot{q}$), vận tốc ($\dot{q}$), và vị trí (q), hệ phương trình vi phân phi tuyến được đưa về dạng ma trận động lực học tổng quát (Động lực học chuẩn):

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau \tag{19}$$

Bằng phương pháp giải tích, các phần tử của 3 ma trận cốt lõi được xác định tường minh như sau:

a. Ma trận khối lượng và quán tính $M(q) \in \mathbb{R}^{2\times 2}$: Đây là ma trận đối xứng, xác định mức độ chống lại sự thay đổi gia tốc của hệ thống.

$$M_{11} = m_1 L_{c1}^2 + I_1 + m_2(L_1^2 + L_{c2}^2 + 2L_1 L_{c2} \cos q_2) + I_2 \tag{20}$$

$$M_{12} = M_{21} = m_2(L_{c2}^2 + L_1 L_{c2} \cos q_2) + I_2 \tag{21}$$

$$M_{22} = m_2 L_{c2}^2 + I_2 \tag{22}$$

b. Ma trận Coriolis và lực ly tâm $C(q, \dot{q}) \in \mathbb{R}^{2\times 2}$: Đại diện cho ảnh hưởng của vận tốc khớp đến lực cản. Đặt hằng số $h = -m_2 L_1 L_{c2} \sin q_2$, ta có:

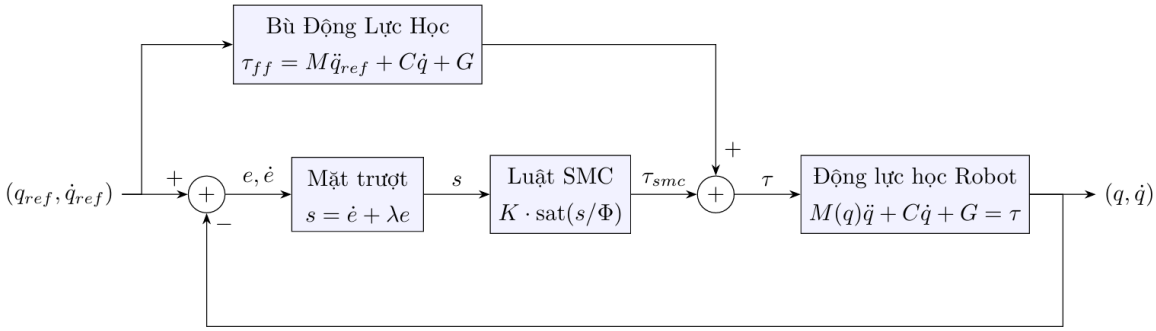
$$C(q, \dot{q}) = \begin{bmatrix} h\dot{q}_2 & h(\dot{q}_1 + \dot{q}_2) \\ -h\dot{q}_1 & 0 \end{bmatrix} \tag{23}$$

c. Vector trọng trường $G(q) \in \mathbb{R}^{2\times 1}$: Đại diện cho mô-men xoắn sinh ra do trọng lực kéo các khâu xuống:

$$G(q) = \begin{bmatrix} (m_1 L_{c1} + m_2 L_1) g \cos q_1 + m_2 L_{c2} g \cos(q_1 + q_2) \\ m_2 L_{c2} g \cos(q_1 + q_2) \end{bmatrix} \tag{24}$$

Các ma trận $M(q)$, $C(q, \dot{q})$ và $G(q)$ này là cơ sở dữ liệu quan trọng, được tính toán trực tiếp theo thời gian thực (online) để thực hiện chức năng bù động lực học (Feedforward Compensation) trong thiết kế bộ điều khiển SMC.

Chương 2: THIẾT KẾ BỘ ĐIỀU KHIỂN TRƯỢT



Hình 3: *Sơ đồ khối bộ điều khiển trượt*

Hình 3 cho thấy vòng điều khiển có hồi tiếp của bộ điều khiển, với từng khối tương ứng từng mục sau.

2.1 Thiết lập mặt trượt

Mục tiêu của bộ điều khiển là bám sát quỹ đạo góc khớp đặt trước. Ta định nghĩa sai số bám (Tracking error) về vị trí và vận tốc cho từng khớp như sau:

$$e(t) = q_{ref}(t) - q(t) \tag{25}$$

$$\dot{e}(t) = \dot{q}_{ref}(t) - \dot{q}(t) \tag{26}$$

Trong đó, q_{ref} và $\dot{q}_{ref}$ lần lượt là vị trí và vận tốc góc khớp mong muốn, q và $\dot{q}$ là các giá trị phản hồi thực tế từ hệ thống.

Mặt trượt tỷ lệ - vi phân (PD) được thiết lập nhằm định nghĩa động học của sai số khi hệ thống tiến về trạng thái ổn định:

$$s(t) = \dot{e}(t) + \lambda e(t) \tag{27}$$

Với $\lambda > 0$ là hằng số dương quyết định độ dốc của mặt trượt. Khi trạng thái hệ thống hội tụ về mặt trượt ($s(t) = 0$), phương trình vi phân bậc nhất $\dot{e}(t) = -\lambda e(t)$ đảm bảo sai số bám $e(t)$ sẽ tiến về 0 theo hàm số mũ.

2.2 Thành phần điều khiển bù động lực học

Dựa trên mô hình động lực học đã xây dựng ở Chương 1:

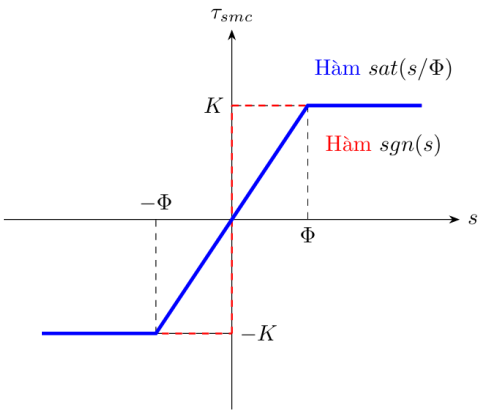
$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau \tag{28}$$

Để cải thiện tốc độ đáp ứng và bù đắp các đặc tính phi tuyến của chân robot, một thành phần mô-men chủ động được tính toán dựa trên động lực học ngược. Thành phần này sử dụng quỹ đạo gia tốc mong muốn $\ddot{q}_{ref}$:

$$\tau_{ff} = M(q)\ddot{q}_{ref} + C(q, \dot{q})\dot{q} + G(q) \tag{29}$$

Khâu bù này đóng vai trò triệt tiêu phần lớn các lực quán tính, lực Coriolis và trọng trường $G(q)$, giúp giảm tải cho khâu điều khiển phản hồi ở các bước sau.

2.3 Thành phần chuyển mạch và giải pháp chống Chattering



Hình 4: *Đồ thị hàm* $\text{sat}\left(\frac{s}{\Phi}\right)$ *và hàm* $\text{sgn}(s)$

Để xử lý các sai số mô hình hóa và nhiễu loạn từ môi trường, luật điều khiển SMC sử dụng thành phần chuyển mạch:

$$\tau_{smc} = K \cdot \text{sgn}(s) \tag{30}$$

Với K là ma trận khuếch đại chuyển mạch. Tuy nhiên, hàm dấu $\text{sgn}(s)$ mang tính gián đoạn, gây ra hiện tượng dao động tần số cao (Chattering) tại tín hiệu ngõ ra. Điều này có thể gây tổn hại nghiêm trọng đến cơ cấu truyền động và động cơ trong thực tế.

Để khắc phục, hàm dấu được thay thế bằng hàm bão hòa (Hình 4 để tạo ra một lớp biên có độ dày Φ xung quanh mặt trượt:

$$\tau_{smc} = K \cdot \text{sat}\left(\frac{s}{\Phi}\right) \tag{31}$$

Trong đó, hàm $\text{sat}(x)$ được định nghĩa:

$$\text{sat}(x) = \begin{cases} x & \text{nếu } |x| \leq 1 \\ \text{sgn}(x) & \text{nếu } |x| > 1 \end{cases} \tag{32}$$

Giải pháp này giúp làm mượt tín hiệu điều khiển τ_{smc} mà vẫn duy trì được độ chính xác bám quỹ đạo trong giới hạn cho phép của lớp biên.

2.4 Tích hợp khâu tích phân và ràng buộc thực tế

Để áp dụng bộ điều khiển lên mô phỏng vật lý Webots, một số thành phần hiệu chỉnh được bổ sung nhằm tối ưu hóa sự ổn định:

- **Khâu Tích phân:** Bổ sung thành phần $\tau_{int} = \int K_i e \, dt$ nhằm triệt tiêu sai số tĩnh do lực ma sát tĩnh hoặc các hằng số chưa được mô hình hóa toàn vẹn.
- **Cơ chế Anti-windup:** Để tránh hiện tượng bão hòa mô-men do khâu tích phân tích lũy quá lớn khi sai số kéo dài, giá trị của τ_{int} được giới hạn cứng trong khoảng $[-\tau_{\text{int_max}}, \tau_{\text{int_max}}]$.

- **Bù ma sát nhớt:** Một thành phần cản nhân tạo $-k_d\dot{q}$ được thêm vào để dập tắt các dao động ký sinh và tăng cường tính ổn định động học.

Luật điều khiển tổng hợp cuối cùng cho mỗi khớp được phát biểu như sau:

$$\tau = \tau_{ff} + \tau_{smc} + \tau_{int} - k_d\dot{q} \tag{33}$$

Sau khi tính toán, tín hiệu τ sẽ được đưa qua khối giới hạn ngõ ra để đảm bảo không vượt quá công suất cực đại của động cơ thực tế.

2.5 Chứng minh ổn định Lyapunov

Sự ổn định của hệ thống được đánh giá thông qua hàm Lyapunov xác định dương:

$$V = \frac{1}{2}s^T M(q)s > 0 \quad (\text{với } s \neq 0) \tag{34}$$

Đạo hàm của hàm Lyapunov theo thời gian:

$$\dot{V} = s^T M(q)\dot{s} + \frac{1}{2}s^T \dot{M}(q)s \tag{35}$$

Ta có đạo hàm của mặt trượt: $\dot{s} = \ddot{e} + \lambda\dot{e} = \ddot{q}_{ref} - \ddot{q} + \lambda\dot{e}$. Thay phương trình động lực học hệ thống $\ddot{q} = M^{-1}(\tau - C\dot{q} - G)$ vào $\dot{s}$, ta thu được:

$$M(q)\dot{s} = M(q)\ddot{q}_{ref} - (\tau - C\dot{q} - G) + M(q)\lambda\dot{e} \tag{36}$$

Thay luật điều khiển τ đã thiết kế ở trên vào hệ thống, và áp dụng tính chất ma trận phản xứng của hệ robot ($\dot{M} - 2C = 0 \implies \frac{1}{2}s^T\dot{M}s = s^TCs$), phương trình đạo hàm Lyapunov được rút gọn thành:

$$\dot{V} = s^T\left(-K \cdot \text{sat}\left(\frac{s}{\Phi}\right) + \Delta d\right) \tag{37}$$

Với Δd đại diện cho các bất định tổng hợp (bao gồm ma sát, nhiễu và phần bù $\lambda\dot{e}$ chưa được triệt tiêu hoàn toàn).

Hệ thống sẽ thỏa mãn điều kiện ổn định $\dot{V} \leq 0$ (tiến dần về mặt trượt) khi và chỉ khi hệ số khuếch đại chuyển mạch K được chọn đủ lớn sao cho:

$$K \geq |\Delta d| + \eta \quad (\text{với } \eta > 0) \tag{38}$$

Điều này chứng minh rằng, với ma trận hệ số K phù hợp, luật điều khiển đề xuất đảm bảo trạng thái hệ thống luôn hội tụ về lớp biên của mặt trượt trong thời gian hữu hạn và duy trì sự ổn định tiệm cận.

Chương 3: QUY HOẠCH DÁNG ĐI VÀ QUỸ ĐẠO

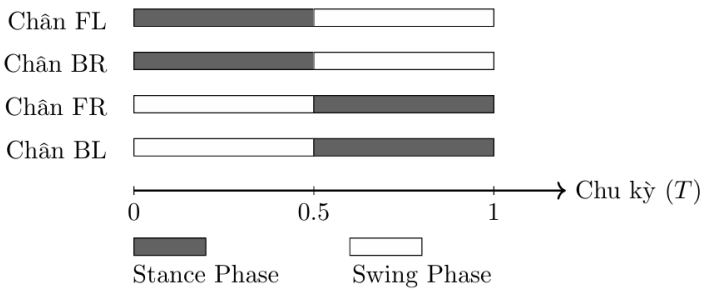
3.1 Thiết kế các dáng đi

Hệ thống điều khiển nhịp điệu của robot dựa trên mạng lưới các bộ dao động (Central Pattern Generators - CPG). Các dáng đi khác nhau được hình thành bằng cách thay đổi ma trận liên kết giữa các bộ dao động này [1].

Dựa trên nguyên lý sinh học, chu kỳ của mỗi chân được đặc trưng bởi góc pha θ. Độ lệch pha giữa các chân quyết định trình tự nhấc chân và chạm đất, từ đó tạo ra các dáng đi đặc trưng:

- **Dáng đi Trot (Kiệu):** Các cặp chân chéo nhau (ví dụ: chân trước trái - FL và chân sau phải - BR) chuyển động hoàn toàn đồng pha, trong khi hai cặp chân chéo này lệch pha nhau đúng nửa chu kỳ ($0.5T$). Đây là dáng đi ổn định ở tốc độ thấp do giảm thiểu vận tốc góc của thân robot [1].
- **Dáng đi Walk (Đi bộ):** Là dáng đi nhịp nhàng trong đó mỗi chân nhấc lên ở một thời điểm khác nhau, tạo thành các khoảng lệch pha liên tiếp là $0.0, 0.5, 0.25, 0.75$ tương ứng cho các chân FL, FR, BR, BL. Dáng đi này được chứng minh là tối ưu nhất về mặt năng lượng (Cost of Transport - COT) ở dải tốc độ thấp (từ $0.3 - 0.9$ m/s) [1].
- **Dáng đi Pace và Bound:** Tương tự, bằng cách cấu hình pha đồng thời cho các chân cùng bên (Pace) hoặc các chân cùng trục trước/sau (Bound), robot có thể linh hoạt chuyển đổi dáng đi ở các dải tốc độ cao hơn [1].

Ở hình 5 là trực quan hóa pha của các chân với dáng đi Trot, trong đó chia làm 2 phase là Swing và Stance, được giải thích ở mục sau.



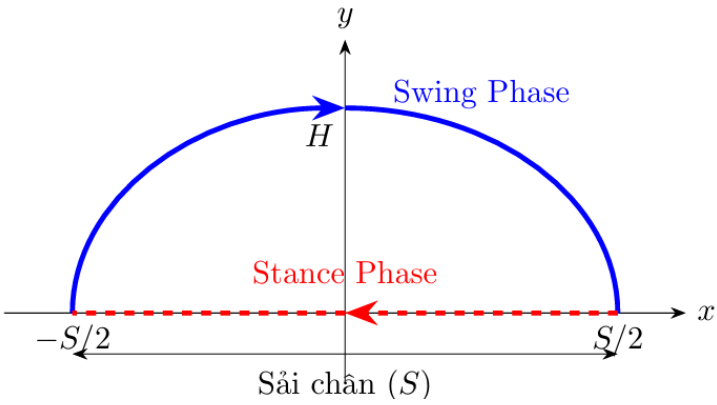
Hình 5: *Biểu đồ pha của các chân với dáng đi Trot*

3.2 Quỹ đạo chân

Trong khi mục 1.2 giải quyết bài toán biến đổi tọa độ không gian thành góc khớp, mục này trình bày cách hệ thống sinh ra tọa độ mục tiêu (x_{ref}, y_{ref}) theo thời gian thực. Thay vì sử dụng các đường cong nội suy phức tạp như Bézier, giải thuật quy hoạch quỹ đạo trong hệ thống này được hiện thực hóa thông qua lớp hình thành mẫu, chuyển đổi trực tiếp góc pha θ_i của mạng CPG thành tọa độ không gian [1].

Như hình 6, quỹ đạo tham chiếu của bàn chân được chia làm hai giai đoạn rõ rệt, kiểm soát bởi các hàm lượng giác dựa trên trạng thái của $\sin(\theta_i)$:

- **Giai đoạn vung chân (Swing phase):** Xảy ra khi $\sin(\theta_i) > 0$ [1]. Bàn chân nhấc khỏi mặt đất tạo thành một quỹ đạo bán elip mượt mà. Tọa độ mục tiêu được tính toán dựa trên độ dài bước



Hình 6: *Quỹ đạo điểm cuối của chân*

dự kiến (Stride - S) và độ cao nhấc chân tối đa (Clearance - H):

$$\begin{cases} x_{ref}(t) = \frac{S}{2}\cos(\theta_i(t)) \\ y_{ref}(t) = y_0 + H\sin(\theta_i(t)) \end{cases} \tag{39}$$

- **Giai đoạn chống chân (Stance phase):** Xảy ra ở nửa chu kỳ còn lại khi $\sin(\theta_i) \leq 0$. Bàn chân bám sát mặt đất để đẩy trọng tâm robot về phía trước, giữ nguyên độ cao thân y_0 [1]:

$$\begin{cases} x_{ref}(t) = -\frac{S}{2}\cos(\theta_i(t) - \pi) \\ y_{ref}(t) = y_0 \end{cases} \tag{40}$$

Thông qua phương pháp này, tín hiệu đặt (x_{ref}, y_{ref}) được cấp liên tục cho bộ giải động học ngược được đề cập ở mục 1.2. Hơn nữa, kiến trúc này cho phép người dùng thay đổi tức thời các thông số phong cách dáng đi (gait style) như chiều cao thân (y_0), độ cao vung chân (H) mà không cần tính toán lại các điểm điều khiển như phương pháp truyền thống [1].

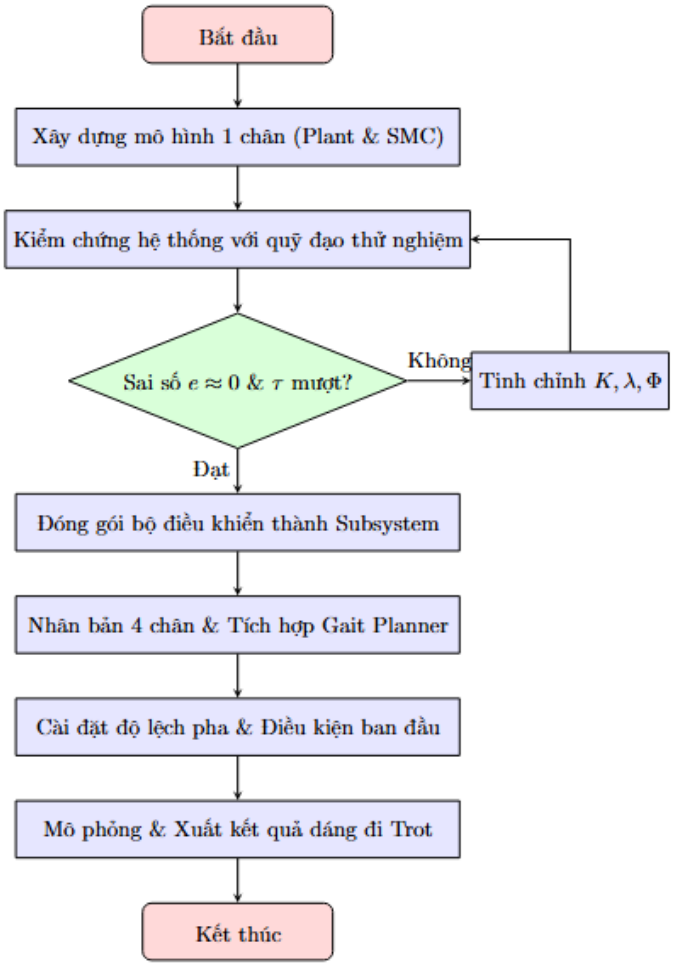
Để đảm bảo tính ổn định, một hệ số thu phóng (scaling factor) được áp dụng trực tiếp lên (x_{ref}, y_{ref}) ở giai đoạn này nếu quỹ đạo yêu cầu vượt quá giới hạn tầm với vật lý của khâu chân $(L_1 + L_2)$, giúp loại bỏ nguy cơ suy biến (singularity) trước khi tín hiệu được đưa vào luật điều khiển SMC.

Chương 4: MÔ PHỎNG VÀ KIỂM CHỨNG TRÊN MATLAB/SIMULINK

Trước khi triển khai thuật toán lên môi trường mô phỏng vật lý 3D (Webots/ROS2), hệ thống được xây dựng và kiểm chứng trên môi trường MATLAB/Simulink. Quá trình này đóng vai trò then chốt nhằm cách ly các yếu tố vật lý không mong muốn (như ma sát tiếp xúc, nhiễu cảm biến), từ đó đánh giá tính chính xác tuyệt đối của phương trình giải tích động lực học và tính hội tụ của luật điều khiển trượt (SMC).

4.1 Xây dựng mô hình mô phỏng trên Simulink

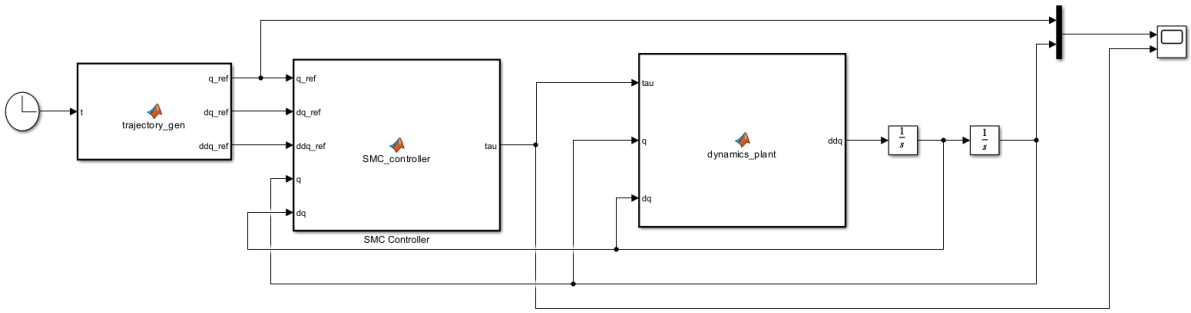
Quy trình xây dựng và kiểm chứng mô phỏng được thực hiện qua hai giai đoạn chính, được minh họa chi tiết trong Lưu đồ Hình **??**. Giai đoạn một tập trung vào kiểm thử đơn vị (Unit Test) cho một nhánh chân đơn lẻ lơ lửng để xác minh hệ phương trình toán học. Sau khi đạt yêu cầu, giai đoạn hai tiến hành đóng gói (Subsystem) và tích hợp quỹ đạo để mô phỏng toàn bộ dáng đi cho hệ thống bốn chân.



Hình 7: *Lưu đồ các bước thiết lập và kiểm chứng mô phỏng trên Simulink*

4.1.1 Kiểm chứng động lực học và SMC trên nhánh chân đơn lẻ

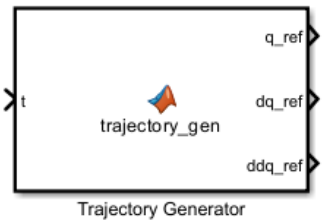
Do robot có kiến trúc cơ khí đối xứng, bốn chân sở hữu các thông số vật lý hoàn toàn giống nhau. Việc kiểm thử trên một nhánh chân giúp phân lập lỗi và xác minh ma trận hệ số một cách độc lập. Sơ đồ Simulink cho một nhánh chân được thiết lập như Hình 8.



Hình 8: *Mô hình Simulink kiểm chứng mô hình động lực học và thuật toán SMC cho 1 chân*

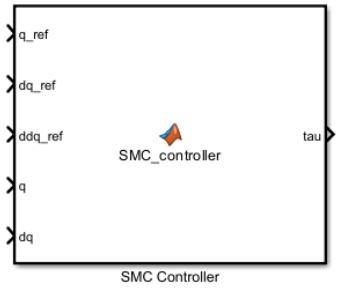
Hệ thống được cấu thành từ 3 khối chức năng (`MATLAB Function`) chính:

- **Khối `trajectory_gen`:** Đóng vai trò sinh quỹ đạo. Trong giai đoạn kiểm chứng này, tín hiệu vị trí mong muốn (q_{ref}) được tạo ra bằng hàm lượng giác tuần hoàn (Sine/Cosine). *Input:* Thời gian t. *Output:* Các vector $q_{ref}, \dot{q}_{ref}, \ddot{q}_{ref} \in \mathbb{R}^{2\times 1}$. Đặc biệt, vận tốc và gia tốc được tính toán trực tiếp bằng đạo hàm giải tích bên trong hàm thay vì dùng khối đạo hàm rời rạc `du/dt` của Simulink, giúp triệt tiêu hoàn toàn nhiễu khuếch đại ở tần số cao.



Hình 9: *Khối tạo quỹ đạo*

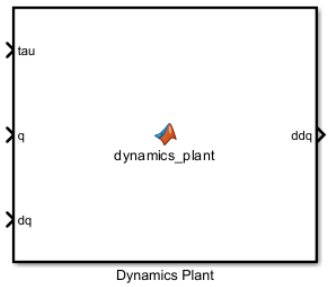
- **Khối `SMC_controller`:** Đóng vai trò là bộ não điều khiển. *Input:* Nhận tín hiệu tham chiếu ($q_{ref}, \dot{q}_{ref}, \ddot{q}_{ref}$) và tín hiệu phản hồi thực tế ($q, \dot{q}$). Khối này tính toán các ma trận M, C, G, xác định mặt trượt s và *Output:* xuất ra vector mô-men xoắn τ đưa tới các khớp.



Hình 10: *Bộ điều khiển SMC*

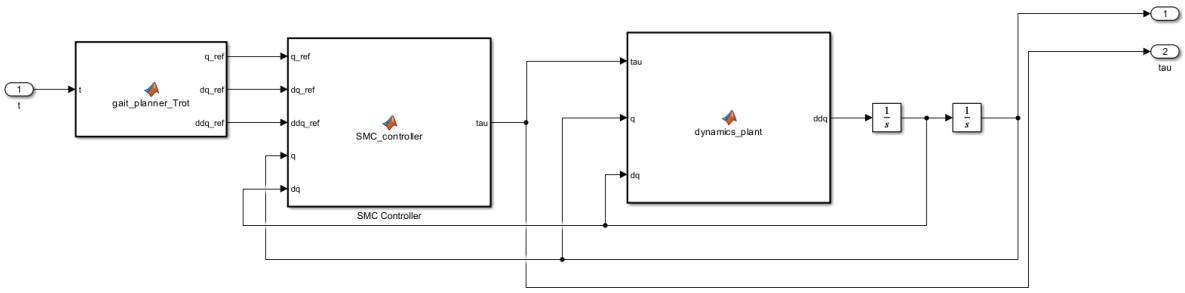
- **Khối `dynamics_plant` & Các khâu tích phân:** Đóng vai trò là mô hình vật lý của chân. *Input:* Lực τ và trạng thái hiện tại $q, \dot{q}$. Khối này giải phương trình $\ddot{q} = M(q)^{-1}(\tau - C\dot{q} - G)$ để *Output:*

xuất ra gia tốc $\ddot{q}$. Hai khối tích phân liên tục ($\frac{1}{s}$) được nối tiếp ngay sau đó để nội suy ra vận tốc $\dot{q}$ và vị trí q thực tế, khép kín vòng điều khiển hồi tiếp.



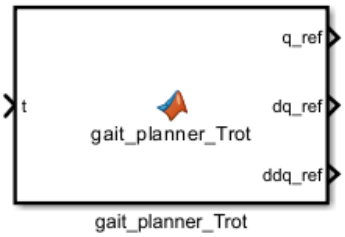
Hình 11: *Đối tượng điều khiển*

4.1.2 Đóng gói hệ thống và tích hợp quỹ đạo Trot cho bốn chân



Hình 12: *Hệ thống con cho 1 chân*

Để mô phỏng chuyển động chạy kiệu (Trot gait) thực tế, khối quỹ đạo thử nghiệm (Sine) bên trong Subsystem được thay thế bằng bộ lập lịch dáng đi kết hợp giải thuật Động học nghịch (Inverse Kinematics). Quỹ đạo bàn chân được thiết kế theo hình chữ D (D-shape trajectory), bao gồm pha nhấc chân (bán elip) và pha chống chân (đường thẳng miết ngang mặt đất).



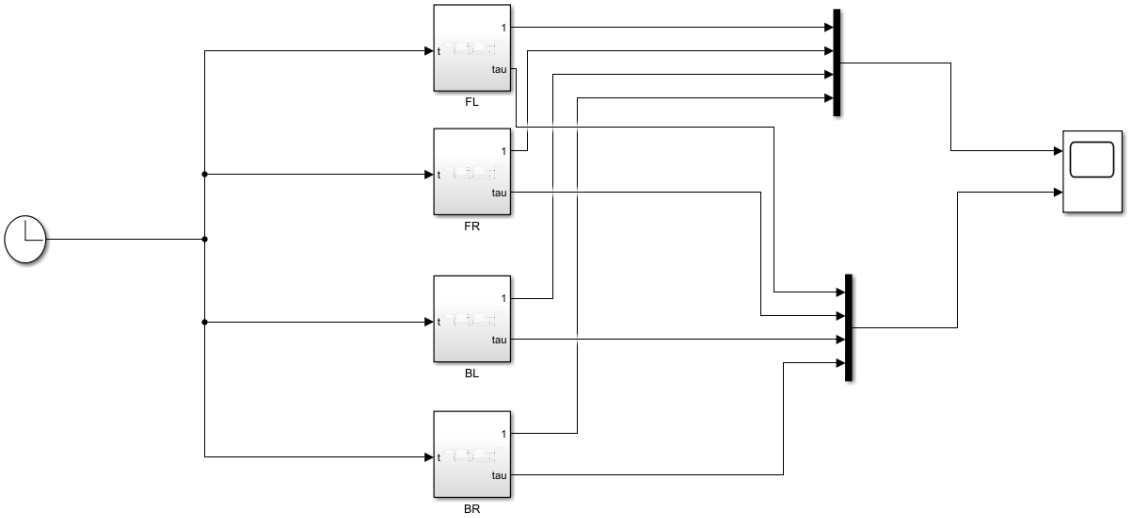
Hình 13: *Khối lập lịch dáng đi*

Cấu trúc bộ tạo quỹ đạo bao gồm hai thành phần chính:

- **Cài đặt lệch pha:** Biến `offset` cho phép điều chỉnh pha dao động của từng chân riêng biệt, định hình nên bản chất của dáng đi Trot.
- **Tính toán vận tốc và gia tốc:** Thay vì sử dụng khối đạo hàm của Simulink (vốn rất nhạy cảm với tín hiệu gãy khúc của quỹ đạo chữ D), thuật toán tiến hành gọi hàm Động học ngược tại các thời điểm quá khứ $t - dt$ và $t - 2dt$. Từ đó, vận tốc $\dot{q}_{ref}$ và gia tốc $\ddot{q}_{ref}$ được tính toán trực tiếp

bằng phương pháp sai phân lùi. Điều này cung cấp tín hiệu tham chiếu cực kỳ ổn định cho bộ điều khiển SMC.

Hệ thống điều khiển nhịp điệu của robot dựa trên mạng lưới các bộ dao động (Central Pattern Generators - CPG). Các dáng đi khác nhau được hình thành bằng cách thay đổi ma trận liên kết giữa các bộ dao động này [1]. Dựa trên nguyên lý đó, khối mô phỏng 1 chân sau khi đạt độ ổn định được tiến hành bôi đen và đóng gói thành hệ thống con (Subsystem - Hình 12). Việc này giúp sơ đồ trở nên gọn gàng trước khi nhân bản thành bốn khối đại diện cho FL, FR, BL, BR (Hình 14).



Hình 14: *Mô hình Simulink kiểm chứng mô hình động lực học và thuật toán SMC cho 1 chân*

Một thiết lập kỹ thuật mang tính quyết định trong mô phỏng toàn hệ thống là cài đặt **Điều kiện ban đầu (Initial Conditions)** cho các khối Tích phân (Integrator) sinh ra vị trí q. Do có sự lệch pha, tại thời điểm $t = 0$, tư thế xuất phát của các chân là hoàn toàn khác nhau. Việc thiết lập góc ban đầu trùng khớp chính xác với góc tính toán từ Động học ngược tại $t = 0$ giúp loại bỏ hoàn toàn các xung lực vọt lố và sai số vị trí khổng lồ ở đầu chu kỳ.

4.2 Kết quả kiểm chứng thuật toán

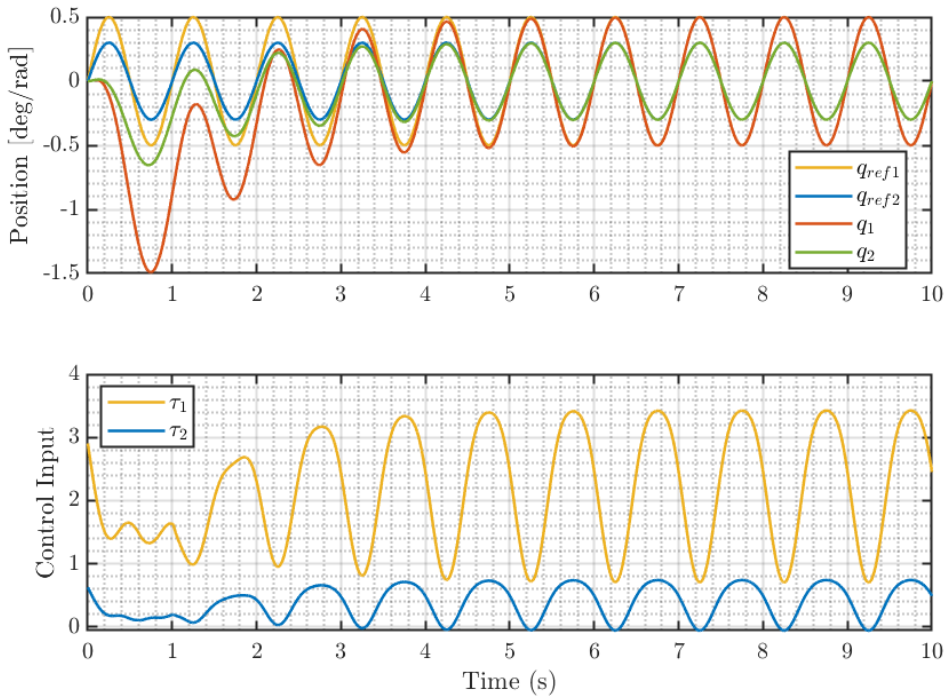
Mục tiêu của trường hợp này là cách ly các vấn đề về quy hoạch dáng đi để tập trung xác minh tính đúng đắn tuyệt đối của phương trình động lực học M, C, G và luật điều khiển SMC bằng một quỹ đạo lượng giác (Sine) cơ bản. Quá trình tinh chỉnh (tuning) bộ điều khiển SMC tập trung vào ma trận hệ số mặt trượt (Λ), độ lợi chuyển mạch (K) và bề dày lớp biên (Φ).

4.2.1 Đáp ứng của nhánh chân đơn lẻ với quỹ đạo thử nghiệm

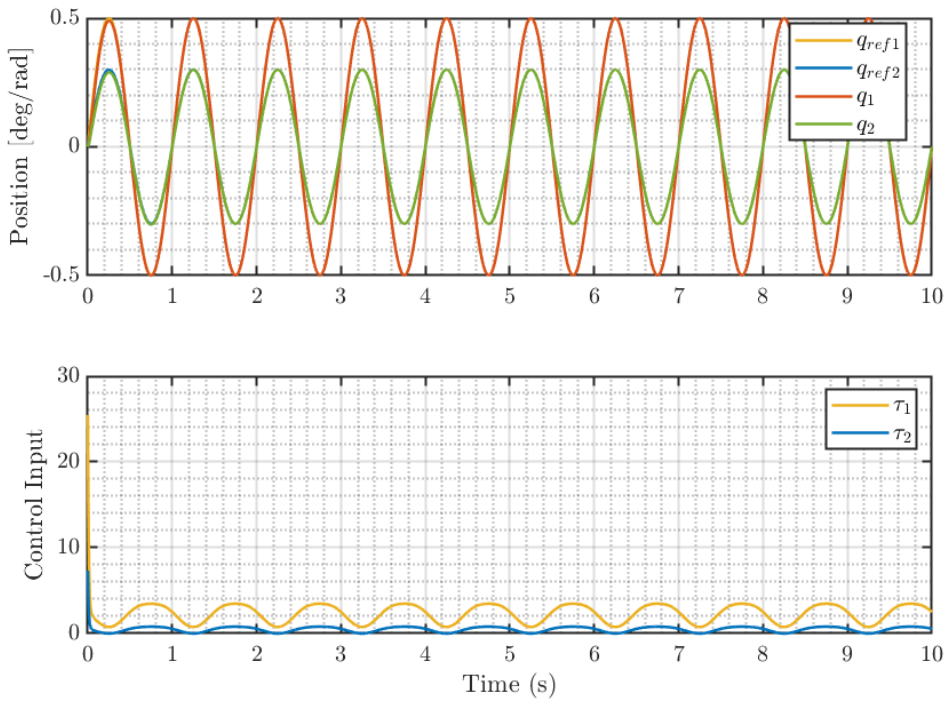
Mục tiêu của trường hợp này là cách ly các vấn đề về quy hoạch dáng đi để tập trung xác minh tính đúng đắn tuyệt đối của phương trình động lực học M, C, G và luật điều khiển SMC bằng một quỹ đạo lượng giác (Sine) cơ bản. Quá trình tinh chỉnh (tuning) bộ điều khiển SMC tập trung vào ma trận hệ số mặt trượt (Λ), độ lợi chuyển mạch (K) và bề dày lớp biên (Φ).

Ban đầu, với các thông số nhỏ ($\lambda = 1$, $K = 3$), hệ thống cho thấy hiện tượng trễ pha; chân robot (đặc biệt là khớp háng phải chịu tải trọng của toàn bộ cẳng chân) mất vài giây để bám được vào quỹ

đạo tham chiếu do không đủ lực kéo (pulling torque) tức thời. Sau quá trình tối ưu hóa, các thông số được tinh chỉnh lên mức $\lambda = 100$, $K_{hip} = 15$, $K_{knee} = 8$ và $\Phi = 3.0$.



(a) *Đáp ứng của bộ điều khiển SMC với các thông số chưa tinh chỉnh*



(b) *Đáp ứng của bộ điều khiển SMC sau khi tinh chỉnh*

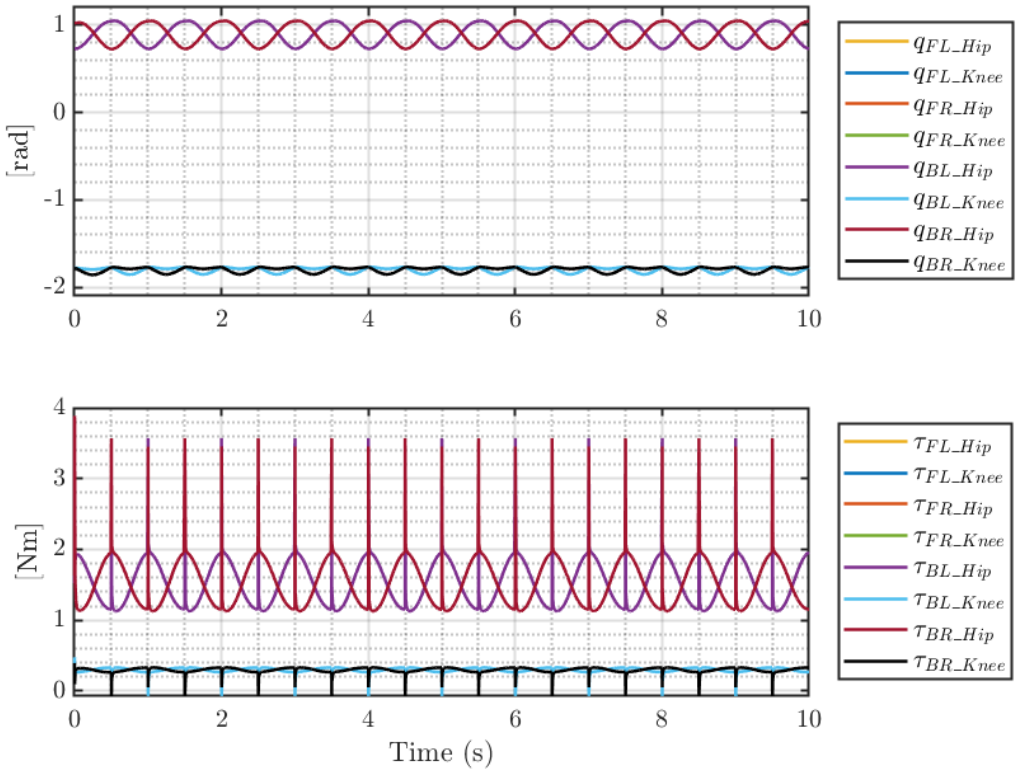
Hình 15: *So sánh đáp ứng bám quỹ đạo của bộ điều khiển SMC trước và sau khi tối ưu thông số*

Dựa trên kết quả thu được từ Scope mô phỏng (Hình 15), các đánh giá kỹ thuật được rút ra như sau:

- **Đồ thị bám quỹ đạo và Đặc tính động học:** Việc áp dụng điều kiện ban đầu (Initial Conditions) chính xác cho các khâu tích phân đã giúp đường tín hiệu vị trí thực tế đè khít hoàn toàn lên đường tín hiệu tham chiếu ngay từ thời điểm $t = 0$. Đồ thị không hề xuất hiện hiện tượng vọt lố (overshoot) hay độ võng ban đầu.
- **Sai số bám (Tracking Error):** Nhờ khâu bù động lực học (Feedforward Compensation) hoạt động chính xác với phương trình giải tích M, C, G, sai số vị trí $e(t)$ hội tụ về 0 gần như tức thời. Điều này chứng minh rằng mô hình toán học xây dựng ở Chương 1 là hoàn toàn chính xác.
- **Phân tích tính ổn định của mặt trượt:** Sự hội tụ của hệ thống được duy trì thông qua thành phần chuyển mạch τ_{smc}. Bằng việc sử dụng hàm bão hòa mềm với lớp biên $\Phi = 3.0$, đồ thị lực điều khiển τ (ngõ ra) vô cùng trơn tru, biến thiên nhịp nhàng theo hình dáng của quỹ đạo vị trí mà không xuất hiện dao động tần số cao (Chattering).

4.2.2 Đáp ứng của hệ thống với dáng đi Trot

Sau khi xác minh thành công ở cấp độ nhánh chân, toàn bộ hệ thống bốn chân được khởi chạy đồng bộ với bộ quy hoạch quỹ đạo Trot gait kết hợp Động học ngược (Inverse Kinematics) để kiểm chứng khả năng phối hợp nhịp điệu. Bên cạnh đó, ta kế thừa những thông số $\lambda = 100$, $K_{hip} = 15$, $K_{knee} = 8$ và $\Phi = 3.0$ đã tìm ở phần 4.2.1.



Hình 16: *Mô hình Simulink kiểm chứng mô hình động lực học và thuật toán SMC cho 1 chân*

Dựa trên kết quả thu được từ cấu hình lệch pha toàn hệ thống (Hình 16), có thể nhận thấy các đặc điểm cơ sinh học phức tạp đã được tái hiện và điều khiển thành công:

- **Đồ thị bám quỹ đạo và Đặc tính động học:** Bốn đồ thị chân chạy song song, đan chéo nhau thành hai cặp (FL-BR và FR-BL) nhịp nhàng không có độ trễ, thể hiện chính xác bản chất của dáng đi Trot. Đặc tính phi tuyến rõ rệt nhất nằm ở khớp gối (q_2), đồ thị có dạng đường cong võng xuống với các giá trị hoàn toàn âm, phản ánh đúng cấu hình cơ học "gập gối ngược" (Knee-in). Sự nhấp nhô liên tục của q_2 trong pha chống chân là minh chứng cho việc thuật toán Động học ngược đang bù trừ độ võng để giữ cho bàn chân di chuyển phẳng dọc theo trục tọa độ.

- **Sai số bám (Tracking Error):** Trong dáng đi Trot, quỹ đạo bàn chân hình chữ D liên tục thay đổi gia tốc (chuyển từ cung tròn sang đường thẳng). Ngay cả tại các điểm gãy khúc mang tính cực đoan này, sai số bám $e(t)$ vẫn bị ép chặt về 0 một cách cưỡng bức. Khả năng theo dõi chính xác các quỹ đạo phi tuyến cường độ cao này khẳng định độ bền vững (robustness) tuyệt đối của thuật toán điều khiển SMC.

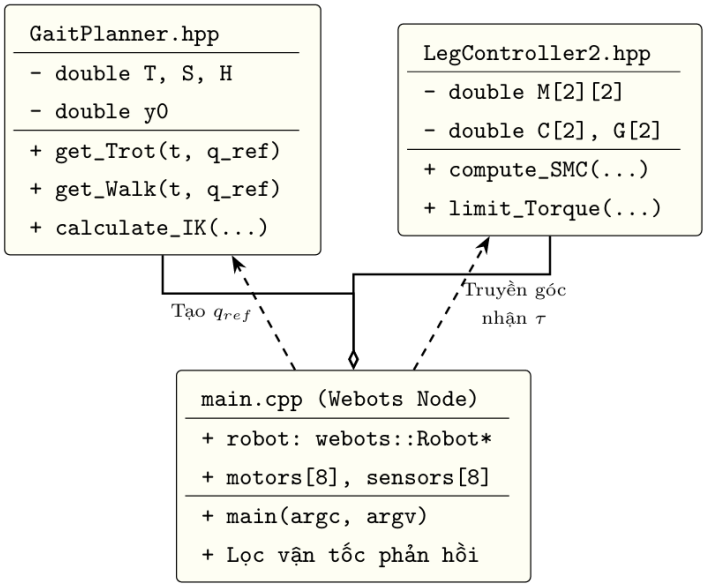
- **Phân tích tính ổn định của mặt trượt:** Sự khác biệt lớn nhất giữa mô phỏng quỹ đạo Sine và quỹ đạo Trot thực tế là sự xuất hiện của các xung lực kéo đẩy. Đồ thị lực τ xuất hiện các xung nhọn (Spike) vọt lên có tính chu kỳ. Đây không phải là hiện tượng mất ổn định (Chattering), mà là phản ứng tức thời của bộ điều khiển để chống lại sự biến thiên gia tốc vô cực tại thời điểm bàn chân robot đập xuống mặt phẳng (chuyển giao giữa pha nhấc chân và pha chống chân). Việc mặt trượt s luôn được duy trì bên trong lớp biên dù chịu lực va đập giả lập chứng tỏ hệ thống đã hoàn toàn sẵn sàng về mặt toán học để triển khai vào môi trường vật lý Webots.

Chương 5: TRIỂN KHAI HỆ THỐNG TRÊN ROS VÀ WEBOTS

5.1 Phát triển bộ điều khiển và tương tác (Webots Standalone)

Giai đoạn này tập trung vào việc xây dựng một hệ thống điều khiển tích hợp trực tiếp bên trong bộ xử lý của Webots. Mục tiêu là tạo ra một mẫu thử có khả năng đáp ứng thời gian thực với tần số cao, đồng thời cho phép người dùng thay đổi trạng thái hoạt động thông qua giao diện bàn phím mà không phải chịu độ trễ từ các middleware bên ngoài.

5.1.1 Kiến trúc hệ thống



Hình 17: *Quỹ đạo điểm cuối của chân*

Như Hình 17, bộ điều khiển được chia thành các đối tượng để dễ dàng quản lý:

- **Lớp Quy hoạch dáng đi (`GaitPlanner.hpp`):** Chịu trách nhiệm tạo ra quỹ đạo tuần hoàn cho các khớp dựa trên chu kỳ thời gian. Lớp này cung cấp các phương thức như `get_Trot`, `get_Walk`, `get_Bound`, `get_Pace` để tính toán tức thời góc khớp mong muốn (q_{ref}) theo phương pháp hình học lượng giác đã thiết kế ở Chương 3.
- **Lớp Điều khiển cấp thấp (`LegController2.hpp`):** Thực hiện tính toán động lực học cho từng chân độc lập (Model-based). Lớp này nhận tín hiệu góc phản hồi từ cảm biến, áp dụng luật điều khiển trượt có bù động lực học để xuất tín hiệu mô-men xoắn ra motor theo từng bước thời gian dt.

5.1.2 Quản lý luồng tín hiệu và Lọc dữ liệu

Để đảm bảo bộ điều khiển SMC hoạt động trơn tru, việc lấy đạo hàm vận tốc cần được xử lý nhiễu. Trong vòng lặp chính của `main.cpp`, hệ thống thực hiện:

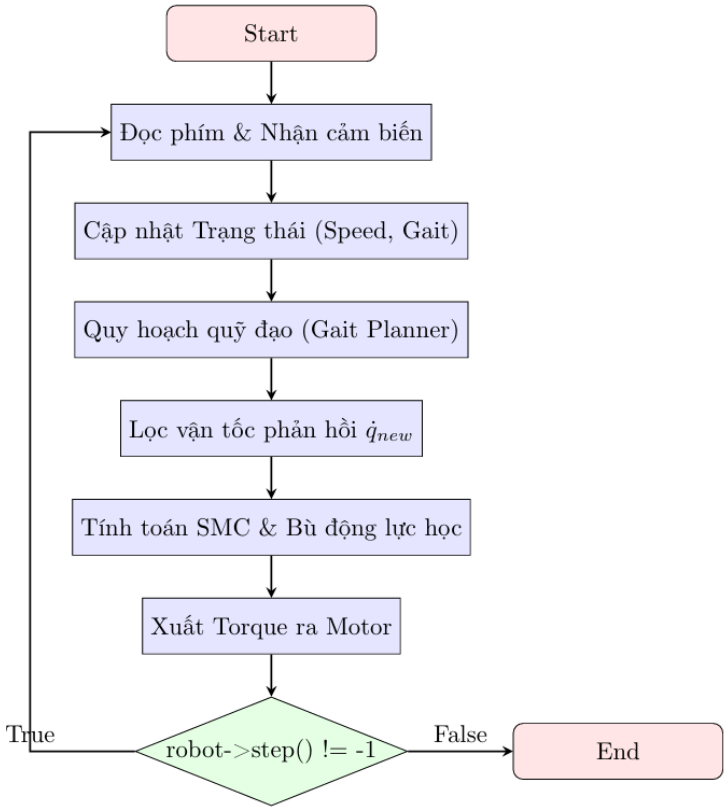
- **Xử lý vận tốc đặt:** Vận tốc tham chiếu $\dot{q}_{ref}$ được tính xấp xỉ qua đạo hàm bậc nhất của q_{ref} theo thời gian, đồng thời giới hạn biên độ ở mức ± 20 rad/s để tránh tín hiệu giật cục.
- **Bộ lọc thông thấp:** Vận tốc thực tế của khớp $\dot{q}$ được tính toán và truyền qua bộ lọc trung bình động hàm số mũ với hệ số $\alpha = 0.15$:

$$\dot{q}_{new} = \alpha \dot{q}_{raw} + (1 - \alpha)\dot{q}_{prev} \tag{41}$$

  Kỹ thuật này giúp loại bỏ gai nhiễu từ vi phân cảm biến vị trí, là điều kiện tiên quyết để hàm $\text{sgn}(s)$ (hoặc hàm bão hòa) trong SMC không bị chattering mạnh.

5.1.3 Xây dựng máy trạng thái và tương tác

Luồng dữ liệu của chương trình được mô tả trong Hình 18. Trong đó, ở trạng thái khi vừa bắt đầu, hệ thống vào giai đoạn khởi tạo ($t < 1.5s$), các motor được cấu hình chạy ở chế độ điều khiển vị trí thuần túy để ép robot vào tư thế chuẩn bị. Sau đó, robot vào giai đoạn điều khiển, dáng đi khởi tạo là Trot và tốc độ là 100%. Sau đó, hệ thống nhận lệnh từ bàn phím (nếu có) và đọc dữ liệu từ cảm biến, tiến hành tính toán và xuất mô-men xoắn ra Motor liên tục.



Hình 18: *Lưu đồ giải thuật của hệ thống*

Một ưu điểm lớn của hệ thống này là khả năng phản hồi thao tác tức thời thông qua thư viện `webots::Keyboard`. Hệ thống hoạt động như một máy trạng thái:

- **Chuyển đổi dáng đi tức thời:** Robot có thể chuyển đổi mượt mà giữa các cấu hình pha (Trot, Walk, Bound, Pace) thông qua phím tắt (T, W, B, P) mà không cần dừng mô phỏng.

- **Điều chỉnh động học thời gian thực:** Biến thời gian `internal_gait_time` được nhân với các hệ số `speed_factor` và `direction` (tiến/lùi) cho phép thay đổi nhịp độ bước chạy từ 20% lên tới 300% ngay trong lúc robot đang di chuyển.

Quá trình này xác nhận mô hình vật lý ổn định và các bộ luật điều khiển C++ đã sẵn sàng để được đóng gói thành các Node trong môi trường ROS.

5.2 Tích hợp hệ thống qua ROS

Sau khi mẫu thử Webots Standalone xác nhận tính ổn định của mô hình vật lý và tính đúng đắn của các bộ luật điều khiển C++, bước tiếp theo là đóng gói toàn bộ hệ thống thành một ROS2 Package có khả năng triển khai trên môi trường phân tán. Kiến trúc ROS2 mang lại nhiều lợi thế quan trọng: chuẩn hóa giao tiếp giữa các thành phần thông qua hệ thống Topics/Servicsubes, khả năng giám sát trạng thái hệ thống theo thời gian thực qua lệnh `ros2 topic echo`, tích hợp dễ dàng với các công cụ trực quan hóa như RViz2, và hỗ trợ mở rộng hệ thống trong tương lai.

Phần này trình bày chi tiết cách tổ chức gói phần mềm, kiến trúc các Node, cơ chế giao tiếp qua các Topics, quy trình biên dịch và khởi chạy toàlịchn hệ thống, cũng như luồng dữ liệu xuyên suốt từ bộ lập lịch dáng đi đến lệnh mô-men xuất ra motor.

5.2.1 Tổ chức gói phần mềm ROS2 (Package Structure)

Toàn bộ hệ thống được tổ chức trong một ROS2 Package duy nhất với tên `quadruped_smc_ros2`. Cấu trúc thư mục tuân theo chuẩn `ament_cmake` của ROS2 Jazzy, bảo đảm khả năng tái sử dụng và di chuyển giữa các máy tính khác nhau:

```
quadruped_smc_ros2/
├── CMakeLists.txt ..................... Cấu hình biên dịch ament_cmake
├── package.xml ........................ Khai báo phụ thuộc ROS2
├── plugin.xml ......................... Khai báo plugin cho pluginlib
├── include/quadruped_smc_ros2/ ........ Header files
│   ├── GaitPlanner.hpp ............... Bộ lập lịch dáng đi & IK
│   ├── LegController2.hpp ............ Bộ điều khiển SMC dựa theo mô hình
│   └── quadruped_driver.hpp .......... Khai báo lớp plugin
├── src/
│   ├── quadruped_driver.cpp .......... Plugin điều khiển chính
│   ├── gait_keyboard_node.cpp ........ Node điều khiển dáng đi qua bàn phím
│   └── error_monitor_node.cpp ........ Node giám sát sai số bám
├── launch/
│   └── robot_launch.py ............... Launch file tích hợp
├── urdf/
│   └── robot_description.urdf ........ Mô tả động học URDF
└── resource/
    └── robot_8dof.wbt ................ File môi trường mô phỏng Webots
```

Hệ thống sử dụng kiến trúc plugin thông qua thư viện `pluginlib` – đây là cơ chế tiêu chuẩn của gói `webots_ros2_driver` để tích hợp các bộ điều khiển tùy chỉnh viết bằng C++ vào vòng điều khiển thời gian thực của Webots mà không cần khởi động một tiến trình Node độc lập. Nhờ đó, bộ điều khiển được

thực thi đồng bộ với mỗi bước mô phỏng ($\Delta t = 2\,\text{ms}$), loại bỏ độ trễ do giao tiếp liên tiến trình (IPC) và bảo đảm tính xác định thời gian (determinism) của vòng điều khiển.

Bảng 4: *Các phụ thuộc chính khai báo trong package.xml*

Gói phụ thuộc	Loại	Vai trò
`rclcpp`	depend	API lõi ROS2 C++
`webots_ros2_driver`	depend	Giao diện plugin–Webots
`pluginlib`	depend	Nạp plugin động tại runtime
`sensor_msgs`	depend	Kiểu tin nhắn `JointState`
`std_msgs`	depend	Kiểu tin nhắn `String`
`robot_state_publisher`	exec_depend	Phát sóng TF từ URDF

5.2.2 Kiến trúc các Node và vai trò

Hệ thống ROS2 bao gồm ba thành phần Node chính với vai trò phân biệt rõ ràng. ba **a) Node điều khiển chính: `QuadrupedDriver` (Plugin)**

`QuadrupedDriver` là lớp C++ triển khai giao diện `webots_ros2_driver::PluginInterface`. Thay vì chạy như một tiến trình Node độc lập, plugin được nạp động vào tiến trình `webots_ros2_driver` thông qua cơ chế `pluginlib`. Điều này có nghĩa là hàm `step()` của plugin được gọi trực tiếp bởi vòng lặp mô phỏng Webots ở mỗi bước thời gian $\Delta t = 2\,\text{ms}$, đảm bảo đồng bộ hoàn toàn giữa bộ điều khiển và trạng thái vật lý.

Plugin được khai báo trong file `plugin.xml` để hệ thống `pluginlib` có thể tra cứu và nạp đúng lớp tại thời điểm chạy:

```
<library path="quadruped_smc_ros2">
  <class type="quadruped_smc_ros2::QuadrupedDriver"
         base_class_type="webots_ros2_driver::PluginInterface">
    <description>Model-Based SMC quadruped controller</description>
  </class>
</library>
```

Lớp plugin chứa hai phương thức bắt buộc theo đặc tả giao diện:

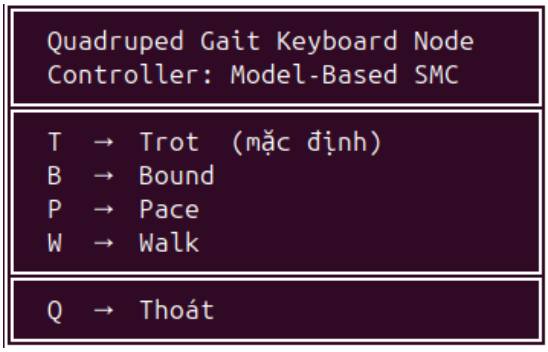
- `init()`: Khởi tạo một lần duy nhất khi hệ thống khởi động. Phương thức này thiết lập kết nối đến 8 motor và 8 cảm biến góc, kích hoạt encoder với chu kỳ lấy mẫu bằng time-step, tạo các Publisher/Subscriber ROS2, và gán trạng thái dáng đi ban đầu là Trot.
- `step()`: Được gọi lặp lại ở mỗi bước mô phỏng. Đây là hàm điều khiển cốt lõi, thực hiện toàn bộ pipeline từ lập lịch quỹ đạo đến xuất lệnh mô-men cho motor (chi tiết tại Mục 5.2.4).

b) Node điều khiển bằng bàn phím: `gait_keyboard_node`

`gait_keyboard_node` là một ROS2 Node tiêu chuẩn (executable) chạy độc lập trong cửa sổ terminal riêng (khởi chạy qua `gnome-terminal` từ launch file). Node thực hiện đọc phím bấm ở chế độ non-blocking thông qua thao tác cấu hình terminal POSIX:

- Thiết lập chế độ raw input bằng cách tắt `ICANON` và `ECHO` trong cấu trúc `termios`.
- Đặt file descriptor stdin thành non-blocking qua `fcntl(STDIN_FILENO, F_SETFL, flags | O_NONBLOCK)`.

- Khôi phục cấu hình terminal về trạng thái ban đầu sau mỗi lần đọc, tránh ảnh hưởng đến các tiến trình khác trên hệ thống.



Hình 19: *Bảng điều khiển gait từ bàn phím*

Node hoạt động ở tần số 50 Hz, liên tục polling bàn phím. Khi người dùng bấm một phím hợp lệ (T, B, P, W, Q), Node xuất một tin nhắn kiểu `std_msgs/String` lên topic `/gait_command`. Cơ chế lọc trùng lặp (chỉ gửi khi dáng đi thực sự thay đổi so với lần gửi trước) giúp giảm tải không cần thiết cho mạng ROS2.

c) Node giám sát sai số bám: `error_monitor_node`

`error_monitor_node` là một ROS2 Node chạy độc lập trong cửa sổ terminal riêng (khởi chạy qua `gnome-terminal` từ launch file), có nhiệm vụ thu thập, phân tích và lưu trữ dữ liệu sai số bám khớp theo thời gian thực.

Node subscribe topic `/joint_states`, trong đó trường `position` chứa góc thực q, trường `effort` chứa góc tham chiếu q_{ref}, và trường `velocity` chứa vận tốc thực $\dot{q}$. Sai số bám tức thời của từng khớp được tính là:

$$e_i = q_{ref,i} - q_i, \quad i = 1, \ldots, 8 \tag{42}$$

Node tích lũy ba chỉ số thống kê cho mỗi khớp theo thời gian hoạt động: sai số bình phương trung bình (RMS), giá trị tuyệt đối trung bình (Mean), và giá trị tuyệt đối cực đại (Max). Kết quả được hiển thị dưới dạng bảng cập nhật 2 Hz trên terminal, đồng thời xuất lên hai topic:

- `/tracking_errors` (`Float64MultiArray`, 8 phần tử): sai số bám tức thời e_i của từng khớp.
- `/tracking_stats` (`String`): chuỗi tóm tắt RMS và Max toàn cục, hỗ trợ giám sát từ xa qua `ros2 topic echo`.

Hệ thống ghi dữ liệu ra hai loại file CSV trong thư mục `~/quadruped_logs/`:

- `joint_data_<timestamp>.csv`: ghi mỗi bước điều khiển một dòng, bao gồm timestamp tương đối (giây), q, q_{ref}, $\dot{q}$ và e của tất cả 8 khớp. File được auto-flush mỗi 500 mẫu (≈1 giây ở 500 Hz) để tránh mất dữ liệu khi tắt đột ngột.
- `stats_summary_<timestamp>.csv`: ghi tóm tắt RMS, Mean, Max từng khớp và chỉ số toàn cục, được tạo tự động khi kết thúc session.

QUADRUPED - TRACKING ERROR MONITOR
Samples: 1892 | R=Reset+NewFile S=Save Q=Quit
Log: druped-Robot/data/joint_data_20260506_223340_717.csv

Joint	Error	RMS	Mean	Max	Flag
FL_hip	0.0258	0.0117	0.0086	0.0434	OK
FL_knee	0.0374	0.0258	0.0174	0.0689	OK
FR_hip	-0.0016	0.0121	0.0091	0.0405	OK
FR_knee	-0.0022	0.0240	0.0174	0.0646	OK
BR_hip	0.0120	0.0098	0.0082	0.0227	OK
BR_knee	0.0440	0.0294	0.0201	0.0794	OK
BL_hip	-0.0203	0.0100	0.0080	0.0267	OK
BL_knee	0.0031	0.0482	0.0341	0.1116	OK

GLOBAL | RMS_all= 0.0248 | Max_all= 0.1116 | Units: rad

Hình 20: *Bảng theo dõi sai số bám của từng khớp*

Node cung cấp ba phím điều khiển hoạt động ở chế độ non-blocking tương tự `gait_keyboard_node`: R để reset thống kê và bắt đầu file CSV mới, S để flush file ngay lập tức không reset, và Q để lưu toàn bộ dữ liệu rồi thoát.

Bảng 5: *Danh sách các Topic ROS2 trong hệ thống*

Tên Topic	Kiểu dữ liệu	Chiều	Mô tả
`/gait_command`	`std_msgs/String`	Pub → Sub	Lệnh chuyển dáng đi (*trot/bound/pace/walk*)
`/joint_states`	`sensor_msgs/JointState`	Driver → All	Vị trí, vận tốc, mô-men khớp (8 DOF)
`/gait_status`	`std_msgs/String`	Driver → All	Dáng đi hiện tại (chỉ phát khi thay đổi)
`/robot_description`	`std_msgs/String`	Launch → All	Chuỗi URDF cho `robot_state_publisher`
`/tracking_errors`	`std_msgs/Float64MultiArray`	Monitor → All	Sai số bám tức thời e_i của 8 khớp (rad)
`/tracking_stats`	`std_msgs/String`	Monitor → All	Chuỗi tóm tắt RMS/Max toàn cục, cập nhật 2 Hz

5.2.3 Hệ thống biên dịch với *ament_cmake*

File `CMakeLists.txt` cấu hình hai target biên dịch độc lập:

a) Thư viện plugin (Shared Library):

Lệnh `add_library()` tạo file `.so` được nạp động tại runtime bởi `pluginlib`. Target này liên kết với toàn bộ các phụ thuộc ROS2 thông qua `ament_target_dependencies()` và được khai báo với `pluginlib_export_plugin_description_file()` để hệ thống có thể định vị `plugin.xml` khi truy vấn.

b) Executable: `gait_keyboard_node`

Lệnh `add_executable()` biên dịch Node bàn phím thành file thực thi độc lập, chỉ phụ thuộc vào `rclcpp` và `std_msgs` – nhẹ nhàng và không kéo theo các phụ thuộc Webots.

Cờ biên dịch `-O2` được bật cho cả hai target để tối ưu hóa hiệu năng tính toán – điều đặc biệt quan trọng với vòng điều khiển 500 Hz. Tiêu chuẩn C++17 được áp dụng để tận dụng các tính năng ngôn ngữ hiện đại.

c) Executable: `error_monitor_node`

Node giám sát được biên dịch thành executable độc lập với phụ thuộc `rclcpp`, `std_msgs` và `sensor_msgs`. Node sử dụng thêm thư viện `<filesystem>` (C++17) để tạo thư mục log tự động tại runtime, và thư viện `<fstream>` để quản lý ghi file CSV theo phiên làm việc.

5.2.4 Luồng dữ liệu và vòng điều khiển

Trong mỗi lần gọi `step()`, `QuadrupedDriver` thực hiện tuần tự bốn giai đoạn xử lý theo pipeline sau:

Giai đoạn 1 – Lập lịch quỹ đạo (Gait Planner)

Bộ `GaitPlanner` tính toán vectơ góc khớp tham chiếu $\mathbf{q}_{\text{ref}} \in \mathbb{R}^8$ dựa trên thời gian dáng đi hiện tại. Trong khoảng 0.5 giây đầu (giai đoạn standup), thời gian dáng đi được đóng băng ở 0 để robot ổn định tư thế trước khi bắt đầu di chuyển. Sau đó, dáng đi được tính theo biến thời gian tương đối:

$$t_{\text{gait}} = t_{\text{elapsed}} - T_{\text{standup}}, \quad T_{\text{standup}} = 0.5\,\text{s}.$$

Bộ lập lịch tra cứu dáng đi hiện tại (`current_gait_` = T/B/P/W) và gọi phương thức tương ứng của `GaitPlanner`.

Giai đoạn 2 – Đọc cảm biến và tính toán đạo hàm

Với mỗi trong 8 khớp, hệ thống đọc giá trị góc thực tế q_i từ encoder thông qua hàm `wb_position_sensor_get_value()`. Vận tốc tham chiếu và gia tốc tham chiếu được xấp xỉ bằng sai phân hữu hạn bậc nhất và bậc hai:

$$\dot{q}_{\text{ref},i} = \frac{q_{\text{ref},i}(k) - q_{\text{ref},i}(k-1)}{\Delta t}, \tag{43}$$

$$\ddot{q}_{\text{ref},i} = \frac{q_{\text{ref},i}(k) - 2\,q_{\text{ref},i}(k-1) + q_{\text{ref},i}(k-2)}{\Delta t^2}. \tag{44}$$

Các giá trị này được giới hạn cứng (clamp) trong phạm vi $|\dot{q}_{\text{ref}}| \leq 20\,\text{rad/s}$ và $|\ddot{q}_{\text{ref}}| \leq 200\,\text{rad/s}^2$ để tránh tín hiệu giật cục khi chuyển đổi dáng đi. Vận tốc thực tế của khớp được ước lượng qua sai phân bậc nhất và được lọc qua bộ lọc thông thấp hàm số mũ (EMA) với hệ số $\alpha = 0.15$ như đã trình bày ở Mục 5.1.2:

$$\dot{q}_{\text{new}} = \alpha\,\dot{q}_{\text{raw}} + (1-\alpha)\,\dot{q}_{\text{prev}}, \tag{45}$$

sau đó được giới hạn trong phạm vi $|\dot{q}| \leq 30\,\text{rad/s}$.

Giai đoạn 3 – Điều khiển Motor (Model-Based SMC)

Giai đoạn này được phân biệt theo trạng thái máy (state machine) hai pha:

- **Pha Standup** ($t < 0.5\,\text{s}$): Motor hoạt động ở chế độ điều khiển vị trí thuần túy bằng cách gọi `wb_motor_set_position()` với $\mathbf{q}_{\text{ref}}$ tĩnh. Điều này ép robot vào tư thế đứng chuẩn bị một cách nhẹ nhàng, tránh hiện tượng sụp đổ do khởi động đột ngột.
- **Pha Điều khiển Mô-men** ($t \geq 0.5\,\text{s}$): Motor được chuyển sang chế độ torque control bằng cách đặt `position = INFINITY` (vô hiệu hóa vòng vị trí nội bộ). Với mỗi chân, `LegController2` tính

toán mô-men điều khiển tổng hợp theo công thức:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{ff}} + \boldsymbol{\tau}_{\text{smc}} + \boldsymbol{\tau}_{\text{int}} - k_d\,\dot{\mathbf{q}}, \tag{46}$$

trong đó $\boldsymbol{\tau}_{\text{ff}}$ là thành phần bù động lực học (feedforward), $\boldsymbol{\tau}_{\text{smc}}$ là thành phần điều khiển trượt với hàm bão hòa chống chattering, $\boldsymbol{\tau}_{\text{int}}$ là thành phần tích phân chống sai số tĩnh có cơ chế anti-windup, và $-k_d\,\dot{\mathbf{q}}$ là thành phần bù ma sát nhớt. Mô-men đầu ra được giới hạn trong khoảng $[-100, +100]$ Nm tương ứng với năng lực tối đa của motor trong mô phỏng.

Giai đoạn 4 – Xuất dữ liệu ROS2

Sau mỗi bước điều khiển, driver xuất trạng thái khớp lên topic `/joint_states` với QoS `SensorDataQoS` (độ trễ thấp, cho phép mất gói). Trường `position` chứa góc đo từ encoder, trường `velocity` chứa vận tốc sau lọc, và trường `effort` được tái sử dụng để mang giá trị góc tham chiếu $\mathbf{q}_{\text{ref}}$ – cho phép theo dõi sai số bám $\mathbf{e} = \mathbf{q}_{\text{ref}} - \mathbf{q}$ trực tiếp qua `ros2 topic echo` mà không cần thêm topic riêng. Dáng đi hiện tại được xuất lên `/gait_status` chỉ khi có sự thay đổi, tiết kiệm băng thông mạng ROS2.

5.2.5 Hệ thống Launch và khởi động tích hợp

File `robot_launch.py` triển khai một `LaunchDescription` tích hợp ba thành phần chính khởi chạy đồng thời, bảo đảm thứ tự phụ thuộc chính xác và tự động dọn dẹp khi kết thúc.

Bảng 6: *Các thành phần trong Launch Description*

Node / Action	Package	Chức năng
`webots_ros2_driver`	`webots_ros2_driver`	Khởi động Webots + nạp plugin `QuadrupedDriver`
`robot_state_publisher`	`robot_state_publisher`	Phát sóng TF cây động học từ URDF + `/joint_states`
`gait_keyboard_node`	`quadruped_smc_ros2`	Cửa sổ terminal điều khiển dáng đi bằng bàn phím
`shutdown_handler`	(Event Handler)	Kết thúc toàn bộ hệ thống khi Webots đóng cửa sổ
`error_monitor_node`	`quadruped_smc_ros2`	Giám sát sai số bám theo thời gian thực

Cơ chế `shutdown_handler` sử dụng `RegisterEventHandler` kết hợp với `OnProcessExit` để bắt sự kiện đóng tiến trình `webots_ros2_driver` và tự động phát `EmitEvent(Shutdown())` đến toàn bộ hệ thống ROS2. Điều này bảo đảm không có tiến trình zombie nào tồn tại sau khi người dùng đóng cửa sổ mô phỏng Webots.

Quy trình biên dịch và khởi chạy được thực hiện qua các lệnh sau:

```
# Bien dich goi phan mem
cd ~/quadruped-robot
colcon build --packages-select quadruped_smc_ros2
source install/setup.bash

# Khoi chay toan he thong robot
ros2 launch quadruped_smc_ros2 robot_launch.py
```

```

# Giam sat trang thai (terminal rieng)
ros2 topic echo /joint_states
ros2 topic echo /gait_status
```

5.2.6 Cơ chế chuyển đổi dáng đi và xử lý sự kiện

Cơ chế chuyển đổi dáng đi được thiết kế để bảo đảm tính liên tục và an toàn. Khi người dùng bấm một phím dáng đi trong cửa sổ `gait_keyboard_node`, quá trình xử lý diễn ra qua ba bước:

1. **Phía Publisher**: `gait_keyboard_node` phát hiện phím bấm qua hàm `getKeyNonBlocking()`. Nếu dáng đi mới khác dáng đi vừa gửi (so sánh với biến `last_gait`), Node tạo tin nhắn `std_msgs::String` và xuất lên `/gait_command` với tên đầy đủ (*"trot"*, *"bound"*, *"pace"*, *"walk"*). Cơ chế lọc này tránh gửi bản tin trùng lặp khi người dùng giữ phím.

2. **Phía Subscriber**: `QuadrupedDriver` tiếp nhận tin nhắn qua callback được đăng ký trong hàm `init()`. Callback trích ký tự đầu tiên của chuỗi, chuyển thành chữ hoa và cập nhật biến `current_gait_` $\in$ {`T, B, P, W`}. Thao tác này là thread-safe trong ROS2 nhờ executor `spin_some()` được gọi ngầm định bởi `webots_ros2_driver`, bảo đảm không có race condition giữa callback subscriber và vòng điều khiển `step()`.

3. **Hiệu ứng trong vòng điều khiển**: Ở lần gọi `step()` kế tiếp, `GaitPlanner` tự động sử dụng dáng đi mới. Sự chuyển đổi là tức thì – không có cơ chế interpolation chuyển tiếp mềm giữa các dáng đi. Tuy nhiên, nhờ thành phần bù động lực học $\boldsymbol{\tau}_{\text{ff}}$ và hàm bão hòa chống chattering trong bộ SMC, các mô-men phát sinh trong giai đoạn chuyển tiếp vẫn được kiểm soát trong giới hạn $\pm 100\,\text{N}\,\text{m}$, tránh được các xung mô-men đột biến gây mất ổn định.

5.2.7 Tổng kết kiến trúc hệ thống

Kiến trúc ROS2 được xây dựng với phương châm phân tách trách nhiệm rõ ràng: `GaitPlanner` chỉ xử lý hình học quỹ đạo, `LegController2` chỉ xử lý tính toán mô-men, `QuadrupedDriver` chỉ điều phối luồng dữ liệu, và `gait_keyboard_node` chỉ xử lý giao tiếp người dùng.

Node `error_monitor_node` bổ sung vai trò kiểm chứng thực nghiệm định lượng cho kiến trúc: toàn bộ sai số bám $e_i(t)$ của 8 khớp được ghi liên tục ra file CSV với tần số bằng tần số mô phỏng (500 Hz), cung cấp dữ liệu thô cho phân tích ở Chương 6. Cụ thể, các chỉ số RMS, Mean và Max được tích lũy online theo công thức:

$$\text{RMS}_i = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_i^2(k)}, \quad \text{Mean}_i = \frac{1}{N}\sum_{k=1}^{N} |e_i(k)|, \quad \text{Max}_i = \max_k |e_i(k)| \tag{47}$$

trong đó N là tổng số mẫu tích lũy trong session. Chỉ số toàn cục được tính là trung bình RMS trên tất cả 8 khớp:

$$\text{RMS}_{\text{global}} = \sqrt{\frac{1}{8}\sum_{i=1}^{8} \text{RMS}_i^2} \tag{48}$$

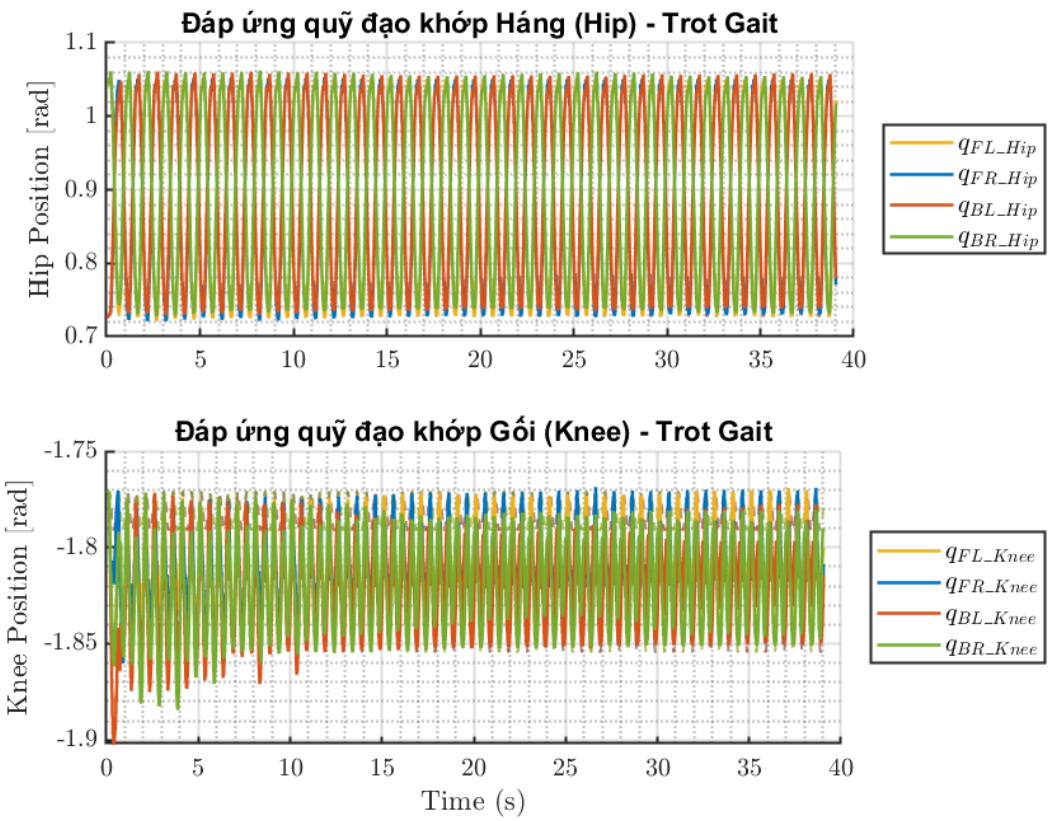
Vì phân tách hoàn toàn khỏi vòng điều khiển, Node này có thể được bật/tắt hoặc thay thế bằng công cụ phân tích khác (rosbag, PlotJuggler) mà không ảnh hưởng đến hoạt động của plugin `QuadrupedDriver`.

Nhờ cấu trúc plugin của `webots_ros2_driver`, vòng điều khiển đạt tần số đồng bộ với mô phỏng Webots (500 Hz tại $\Delta t = 2\,\text{ms}$), trong khi hệ thống ROS2 vẫn duy trì đầy đủ khả năng giám sát, ghi log và mở rộng đặc trưng của nền tảng robot phân tán. Toàn bộ kiến trúc này tạo nền tảng vững chắc cho các nghiên cứu tiếp theo: tích hợp cảm biến ngoại cảnh (camera depth, LiDAR), thêm thuật toán lập kế hoạch đường đi (navigation stack), hoặc triển khai bộ điều khiển học tăng cường thay thế cho bộ SMC hiện tại – tất cả đều có thể thực hiện bằng cách thêm Node mới và đăng ký các Topic tương ứng mà không cần thay đổi cấu trúc plugin cốt lõi.

Chương 6: KẾT QUẢ VÀ THẢO LUẬN

6.1 Kết quả bám quỹ đạo bàn chân

6.1.1 Dáng đi Trot (Đi nước kiệu)



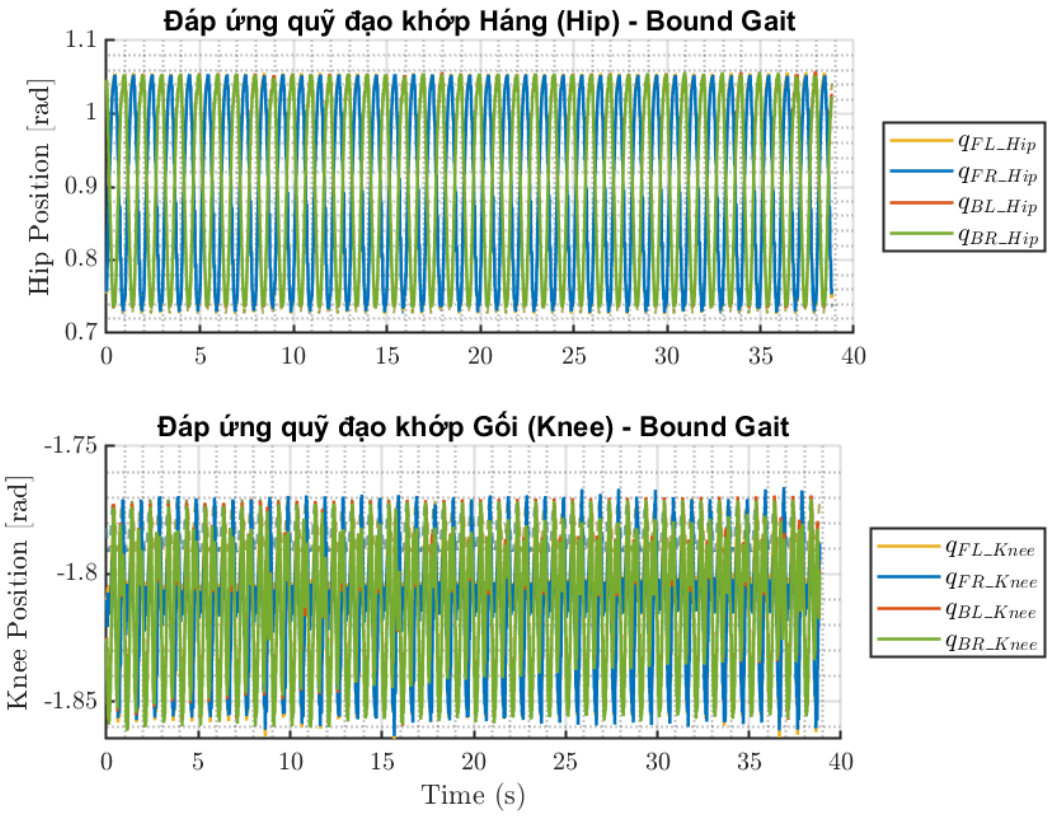
Hình 21: *Đáp ứng bám quỹ đạo của dáng đi Walk*

6.1.2 Dáng đi Bound (Phi ngựa)

- **Khớp háng:** Góc vị trí các khớp dao động trong dải xấp xỉ $[0.75, 1.05]$ rad với chu kỳ đều đặn trong suốt 40 giây mô phỏng. Đặc trưng cơ sinh học của Bound được thể hiện rõ qua sự phân nhóm của các đường tín hiệu: q_{FL_hip} và q_{FR_hip}. Chồng lên nhau gần như hoàn toàn, phản ánh cặp chân trước di chuyển đồng pha; tương tự, q_{BR_hip} và q_{BL_hip} tạo thành cặp thứ hai lệch pha $0.5T$ so với cặp đầu. Hệ thống không xuất hiện hiện tượng trôi (drift) hay suy giảm biên độ theo thời gian, xác nhận sự ổn định tiệm cận của vòng điều khiển.

- **Khớp gối:** Góc khớp gối dao động trong dải xấp xỉ $[-1.87, -1.75]$ rad – hoàn toàn âm, phù hợp với cấu hình cơ học "gập gối ngược" để trung của robot bốn chân. Biên độ dao động của khớp gối hẹp hơn (0.12 rad) so với khớp háng (0.30 rad), điều này là hợp lý về mặt động học vì khớp gối chủ yếu đóng vai trò hấp thụ va đập và duy trì độ cao thân. Đáng chú ý, tín hiệu của bốn khớp gối gần như trùng nhau hoàn toàn, cho thấy bộ Động học ngược (IK) đang hoạt động chính xác trong việc chuyển đổi quỹ đạo bàn chân sang không gian khớp.

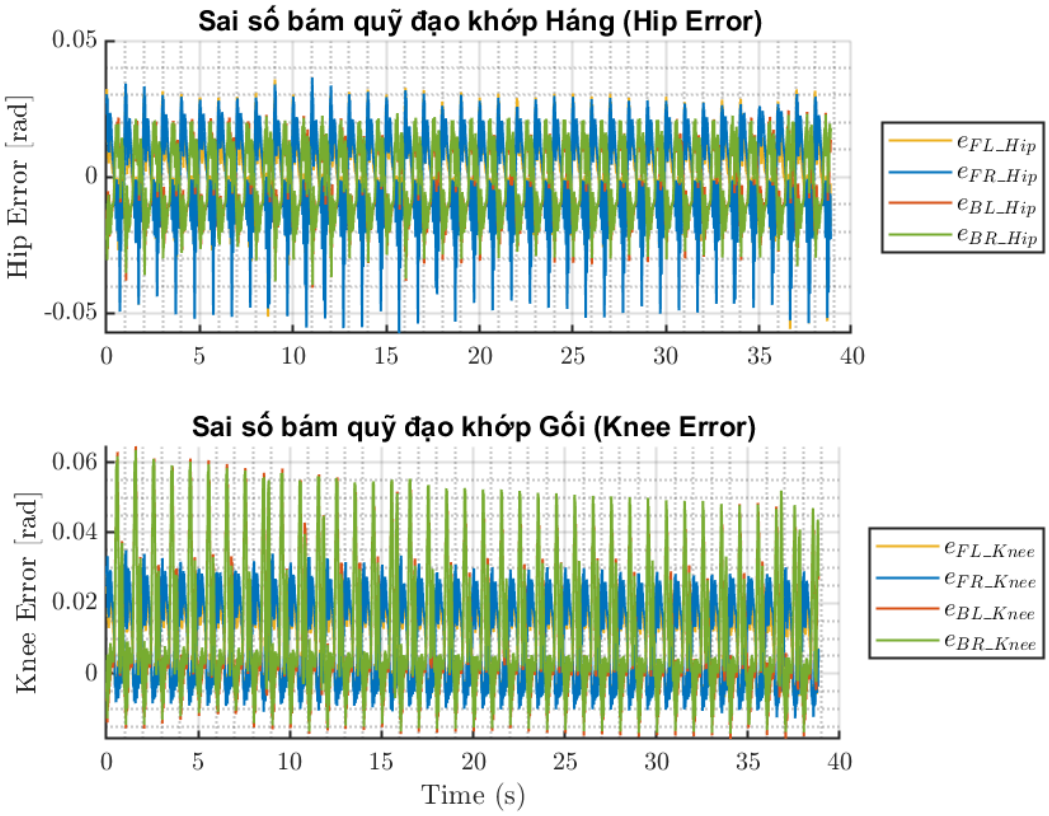
Sai số bám



Hình 22: *Đáp ứng bám quỹ đạo của dáng đi Bound*

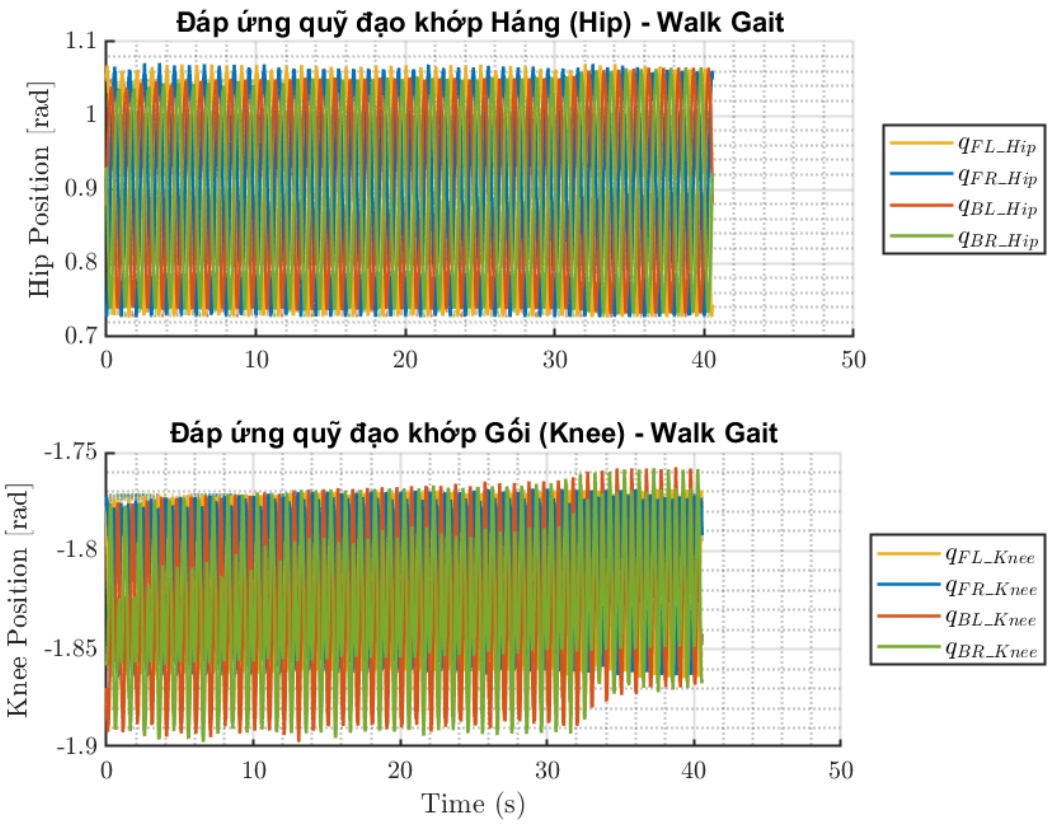
- **Khớp háng:** $e_{hip}(t)$ duy trì trong biên độ $\pm 0.050.05 \pm 0.05$rad trong toàn bộ quá trình mô phỏng 40 giây. Sai số có tính tuần hoàn đồng bộ với chu kỳ bước chân, không có xu hướng tăng trưởng theo thời gian — đây là bằng chứng định lượng cho tính ổn định bền vững của bộ SMC. Thành phần bù động lực học đóng vai trò trung tâm trong việc triệt tiêu phần lớn lực quán tính và trọng trường, giữ cho sai số trong biên ± 0.05 rad ngay cả khi robot chịu tải động lớnở pha chuyển giao.

- **Khớp gối:** $e_{knee}(t)$ nằm chủ yếu trong dải [0, 0.06] rad, mang giá trị dương, phản ánh một lệch hệ thống nhỏ về phía gập gối. Điều này có thể được lý giải bởi tính bất đối xứng của quỹ đạo chữ D tại pha chống chân (Stance): khi bàn chân tiếp đất, lực phản hồi tác động nhanh tạo ra sai số tức thời trước khi thành phần tích phânint$\tau_{int}intkpbtr.Tuynhin, gitrccØikhngvtqu0.06rad\breve{}nhhn3.5\breve{}lmcchpnhnØctrongthctvnhnh.$



Hình 23: *Sai số bám của dáng đi Bound*

6.1.4 Dáng đi Walk (Đi bộ)



Hình 24: *Đáp ứng bám quỹ đạo của dáng đi Walk*

6.2 Phân tích độ ổn định thân Robot

sub Đánh giá dữ liệu từ cảm biến IMU (Roll, Pitch, Yaw) để xác định độ cân bằng của thân robot trong quá trìsubnh di chuyển. Phân tích sự ảnh hưởng của tần số bước đi đến độ ổn định.

6.3 Đánh giá tính bền vững với nhiễu hệ thống

Kết quả thực nghsubiệm khi thay đổi các thông số môi trường hoặc tác động ngoại lực vào robot. Chứng minh khả năng kháng nhiễu của bộ điều khiển SMC.

LỜI KẾT

Báo cáo này đã trình bày toàn bộ quá trình nghiên cứu, thiết kế và triển khai hệ thống điều khiển cho robot bốn chân, từ nền tảng lý thuyết động học đến mô phỏng thực tiễn. Hệ thống đã xây dựng thành công phương trình động lực học Euler-Lagrange và giải quyết bài toán động học ngược để quy hoạch các dáng đi đa dạng như Trot, Walk, Pace và Bound dựa trên mạng lưới bộ dao động CPG. Việc áp dụng bộ điều khiển trượt (SMC) kết hợp thành phần bù động lực học và hàm bão hòa đã chứng minh được tính hiệu quả cao trong việc triệt tiêu hiện tượng dao động tần số cao (*chattering*), giảm thiểu sai số bám và đảm bảo độ bền vững cho hệ thống khi hoạt động ở các quỹ đạo phi tuyến. Quá trình phát triển được thực hiện bài bản qua hai giai đoạn: kiểm chứng giải tích độc lập trên môi trường MATLAB/Simulink và triển khai tích hợp trực quan trong Webots. Đặc biệt, việc đóng gói toàn bộ thuật toán thành các Node ROS2 sử dụng ngôn ngữ C++ đã đáp ứng hoàn hảo yêu cầu xử lý thời gian thực, đồng thời tối ưu hóa hiệu năng tương tác firmware trực tiếp tại cấp thấp. Kiến trúc plugin của `webots_ros2_driver` giúp hệ thống đạt được sự đồng bộ tuyệt đối với chu kỳ mô phỏng ở tần số 500 Hz, tạo ra một kiến trúc điều khiển phân tán vững chắc, linh hoạt và không có độ trễ. Mặc dù hệ thống đã đạt được những kết quả khả quan về đáp ứng bám quỹ đạo và khả năng chuyển đổi trạng thái tức thời, hướng nghiên cứu này vẫn còn nhiều không gian để phát triển. Nền tảng ROS2 hiện tại đã sẵn sàng để tích hợp thêm hệ thống cảm biến ngoại cảnh như camera chiều sâu hay LiDAR, làm cơ sở cho các thuật toán lập kế hoạch đường đi (*navigation stack*), hoặc mở rộng sang các phương pháp điều khiển học tăng cường (*Reinforcement Learning*) nhằm tối ưu hóa sự thích nghi của robot trong môi trường địa hình phức tạp. Nhóm thực hiện

xin gửi lời cảm ơn chân thành đến PGS.TS Nguyễn Tấn Lũy đã tận tình giảng dạy, hướng dẫn và định hướng trong suốt quá trình học tập môn Lý thuyết Điều khiển nâng cao. Những kiến thức nền tảng vững chắc cùng sự hỗ trợ quý báu từ thầy là động lực và cơ sở quan trọng giúp nhóm hoàn thành xuất sắc bài tập lớn này.

TÀI LIỆU THAM KHẢO

[1] G. Bellegarda, M. Shafiee, and A. Ijspeert, "Allgaits: Learning all quadruped gaits and transitions," 2024. [Online]. Available: https://arxiv.org/abs/2411.04787